

```
In [1]: # Local PyCharm / Jupyter setup
        from local_setup import prepare
        prepare()
```

Project: E:\DeepLearning\111\assignment1_colab\assignment1
Python: D:\Wei\python_3.12\python.exe
CIFAR-10 ready.

k-Nearest Neighbor (kNN) exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transferring the labels of the k most similar training examples
- The value of k is cross-validated

In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

```
In [2]: # Run some setup code for this notebook.

import random
import numpy as np
np.random.seed(0)
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the notebook
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

```
In [3]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause memory issue)
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
```

```
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

E:\Deeplearning\111\assignment1_colab\assignment1\cs231n\data_utils.py:16: VisibleDeprecationWarning: dtype() align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)

```
return pickle.load(f, encoding="latin1")
```

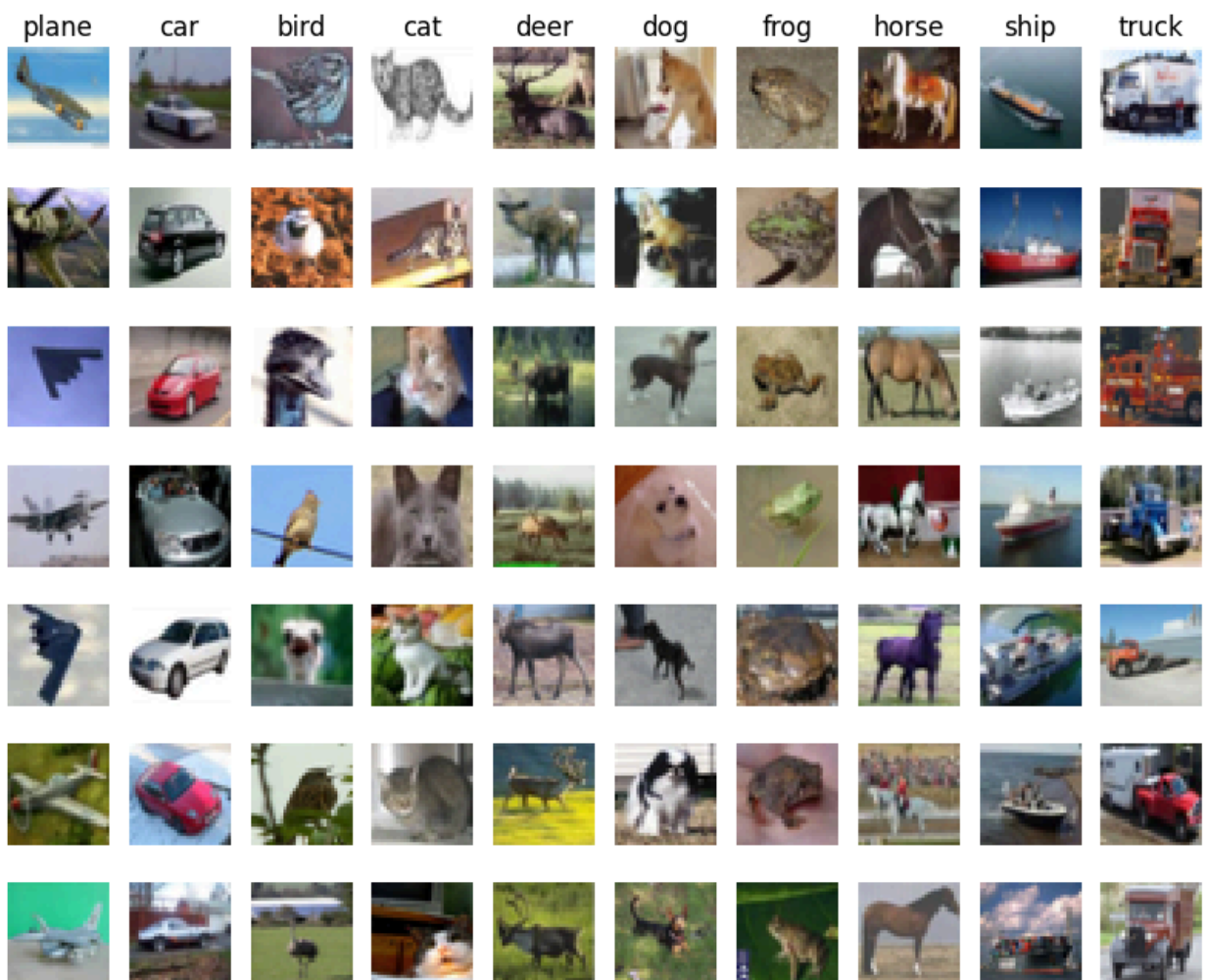
Training data shape: (50000, 32, 32, 3)

Training labels shape: (50000,)

Test data shape: (10000, 32, 32, 3)

Test labels shape: (10000,)

```
In [4]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



```
In [5]: # Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]
```

```

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

# Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)

```

(5000, 3072) (500, 3072)

```

In [6]: from cs231n.classifiers import KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)

```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are **Ntr** training examples and **Nte** test examples, this stage should result in a **Nte x Ntr** matrix where each element (i,j) is the distance between the i-th test and j-th train example.

Note: For the three distance computations that we require you to implement in this notebook, you may not use the `np.linalg.norm()` function that numpy provides.

First, open `cs231n/classifiers/k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

```

In [7]: # Open cs231n/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)

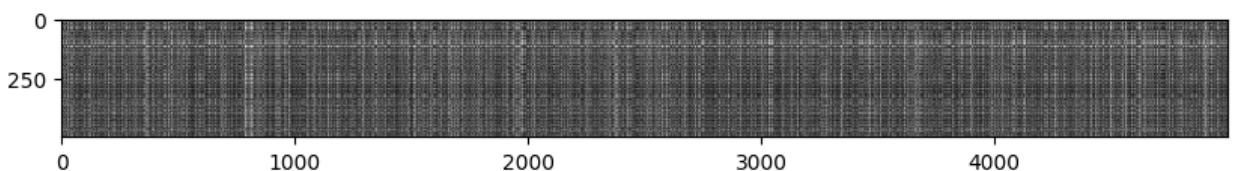
```

(500, 5000)

```

In [8]: # We can visualize the distance matrix: each row is a single test example and
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()

```



Inline Question 1

Notice the structured patterns in the distance matrix, where some rows or columns are visibly brighter. (Note that with the default color scheme black indicates low distances while white indicates high distances.)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

Answer: A bright row corresponds to a test image far from most training images in raw-pixel L2 distance. A bright column corresponds to a training image far from most test images. Unusual overall brightness, color, or pixel patterns can cause this; the distance matrix alone does not identify a unique cause.

```
In [9]: # Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 137 / 500 correct => accuracy: 0.274000

You should expect to see approximately 27% accuracy. Now let's try out a larger `k`, say `k = 5`:

```
In [10]: y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

Got 139 / 500 correct => accuracy: 0.278000

You should expect to see a slightly better performance than with `k = 1`.

Inline Question 2

We can also use other distance metrics such as L1 distance. For pixel values $p_{ij}^{(k)}$ at location (i, j) of some image I_k ,

the mean μ across all pixels over all images is

$$\mu = \frac{1}{nhw} \sum_{k=1}^n \sum_{i=1}^h \sum_{j=1}^w p_{ij}^{(k)}$$

And the pixel-wise mean μ_{ij} across all images is

$$\mu_{ij} = \frac{1}{n} \sum_{k=1}^n p_{ij}^{(k)}.$$

The general standard deviation σ and pixel-wise standard deviation σ_{ij} is defined similarly.

Which of the following preprocessing steps will not change the performance of a Nearest Neighbor classifier that uses L1 distance? Select all that apply. To clarify, both training and test examples are preprocessed in the same way.

1. Subtracting the mean μ ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu$.)
2. Subtracting the per pixel mean μ_{ij} ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu_{ij}$.)
3. Subtracting the mean μ and dividing by the standard deviation σ .
4. Subtracting the pixel-wise mean μ_{ij} and dividing by the pixel-wise standard deviation σ_{ij} .
5. Rotating the coordinate axes of the data, which means rotating all the images by the same angle. Empty regions in the image caused by rotation are padded with a same pixel value and no interpolation is performed.

Answer: Choices 1, 2, and 3 (assuming a positive, nonzero standard deviation).

Subtracting the same scalar or per-pixel mean from both images cancels in their difference. Dividing every coordinate by one common positive standard deviation scales all L1 distances equally, preserving neighbor order. Choice 4 reweights coordinates differently and can change that order. A general rotation does not preserve L1 distance; raster rotation can also lose or duplicate pixels. A special lossless pixel permutation, such as a 90-degree rotation of a square image, is an exception, but choice 5 is not guaranteed for arbitrary angles.

```
In [11]: # Now Lets speed up distance matrix computation by using partial vectorization
# with one loop. Implement the function compute_distances_one_loop and run the
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that it
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words, reshape
# the matrices into vectors and compute the Euclidean distance between them.
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('One loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

One loop difference was: 0.000000
Good! The distance matrices are the same

```
In [12]: # Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)

# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('No loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

No loop difference was: 0.000000
Good! The distance matrices are the same

```
In [13]: # Let's compare how fast the implementations are
def time_function(f, *args):
    """
    Call a function f with args and return the time (in seconds) that it took to execute.
    """
    import time
    tic = time.time()
    f(*args)
```

```

    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print('No loop version took %f seconds' % no_loop_time)

# You should see significantly faster performance with the fully vectorized implementation!

# NOTE: depending on what machine you're using,
# you might not see a speedup when you go from two loops to one loop,
# and might even see a slow-down.

```

Two loop version took 20.743865 seconds
 One loop version took 42.057287 seconds
 No loop version took 0.456310 seconds

Cross-validation

We have implemented the k-Nearest Neighbor classifier but we set the value $k = 5$ arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

```

In [14]: num_folds = 5
k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]
X_train_folds = np.array_split(X_train, num_folds)
y_train_folds = np.array_split(y_train, num_folds)
k_to_accuracies = {k: [] for k in k_choices}

# Reuse each fold's distances across k values; never tune on the test set.
for fold in range(num_folds):
    train_X = np.concatenate([X_train_folds[j] for j in range(num_folds) if j != fold])
    train_y = np.concatenate([y_train_folds[j] for j in range(num_folds) if j != fold])
    fold_classifier = KNearestNeighbor()
    fold_classifier.train(train_X, train_y)
    fold_dists = fold_classifier.compute_distances_no_loops(X_train_folds[fold])
    for k in k_choices:
        predictions = fold_classifier.predict_labels(fold_dists, k=k)
        k_to_accuracies[k].append(np.mean(predictions == y_train_folds[fold]))

for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))

```

```

k = 1, accuracy = 0.263000
k = 1, accuracy = 0.257000
k = 1, accuracy = 0.264000
k = 1, accuracy = 0.278000
k = 1, accuracy = 0.266000
k = 3, accuracy = 0.239000
k = 3, accuracy = 0.249000
k = 3, accuracy = 0.240000
k = 3, accuracy = 0.266000
k = 3, accuracy = 0.254000
k = 5, accuracy = 0.248000
k = 5, accuracy = 0.266000
k = 5, accuracy = 0.280000
k = 5, accuracy = 0.292000
k = 5, accuracy = 0.280000
k = 8, accuracy = 0.262000
k = 8, accuracy = 0.282000
k = 8, accuracy = 0.273000
k = 8, accuracy = 0.290000
k = 8, accuracy = 0.273000
k = 10, accuracy = 0.265000
k = 10, accuracy = 0.296000
k = 10, accuracy = 0.276000
k = 10, accuracy = 0.284000
k = 10, accuracy = 0.280000
k = 12, accuracy = 0.260000
k = 12, accuracy = 0.295000
k = 12, accuracy = 0.279000
k = 12, accuracy = 0.283000
k = 12, accuracy = 0.280000
k = 15, accuracy = 0.252000
k = 15, accuracy = 0.289000
k = 15, accuracy = 0.278000
k = 15, accuracy = 0.282000
k = 15, accuracy = 0.274000
k = 20, accuracy = 0.270000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.282000
k = 20, accuracy = 0.285000
k = 50, accuracy = 0.271000
k = 50, accuracy = 0.288000
k = 50, accuracy = 0.278000
k = 50, accuracy = 0.269000
k = 50, accuracy = 0.266000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.270000
k = 100, accuracy = 0.263000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.263000

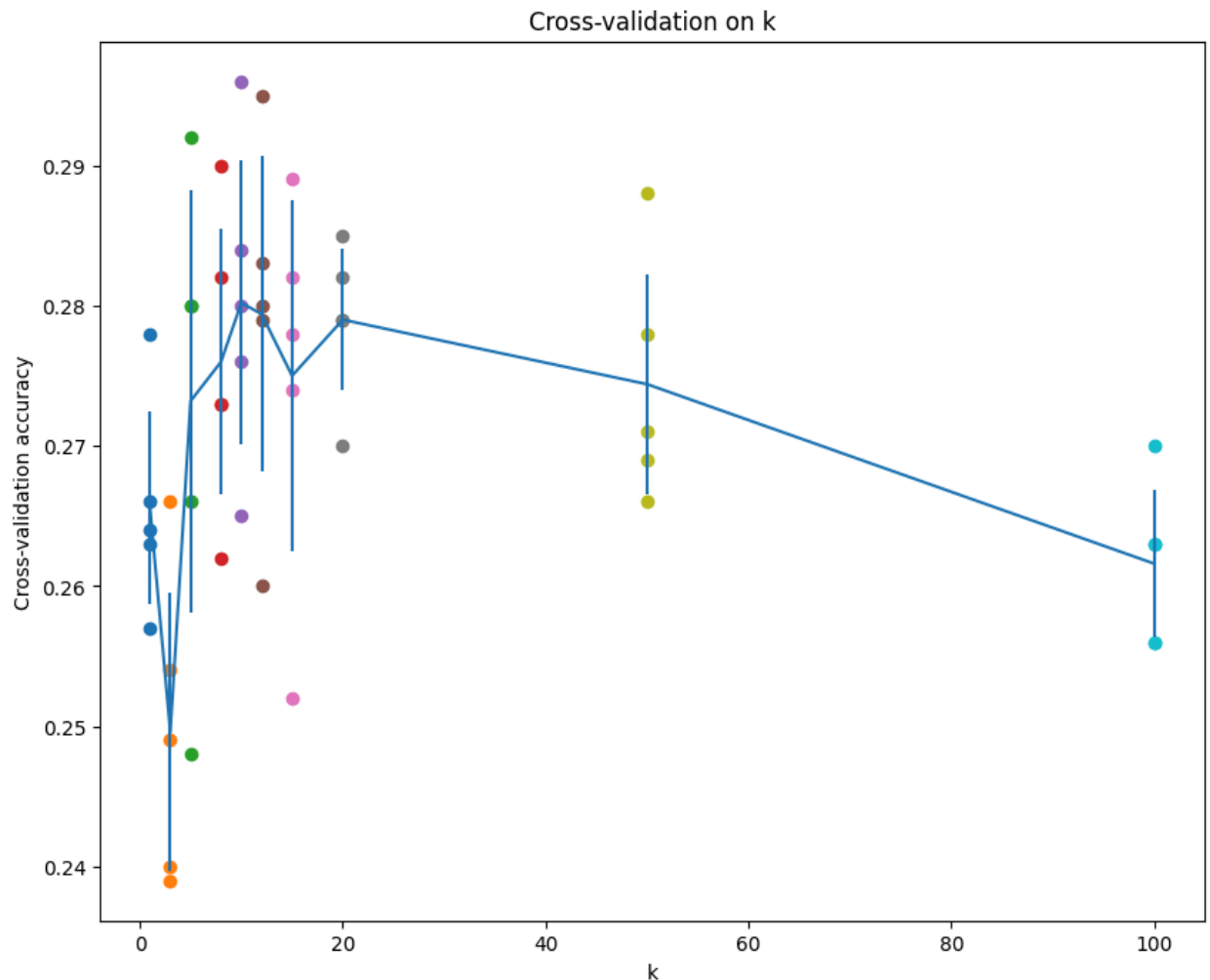
```

```

In [15]: # plot the raw observations
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)

# plot the trend line with error bars that correspond to standard deviation
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.items())])
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.items())])
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
plt.title('Cross-validation on k')
plt.xlabel('k')
plt.ylabel('Cross-validation accuracy')
plt.show()

```



```
In [16]: # Based on the cross-validation results above, choose the best value for k,
# retrain the classifier using all the training data, and test it on the test
# data. You should be able to get above 28% accuracy on the test data.
best_k = max(sorted(k_choices), key=lambda k: np.mean(k_to accuracies[k]))
print('Best k from cross-validation:', best_k)

classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)

# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Best k from cross-validation: 10
Got 141 / 500 correct => accuracy: 0.282000
```

Inline Question 3

Which of the following statements about k -Nearest Neighbor (k -NN) are true in a classification setting, and for all k ? Select all that apply.

1. The decision boundary of the k -NN classifier is linear.
2. The training error of a 1-NN will always be lower than or equal to that of 5-NN.
3. The test error of a 1-NN will always be lower than that of a 5-NN.
4. The time needed to classify a test example with the k -NN classifier grows with the size of the training set.
5. None of the above.

Answer: Choices 2 and 4 under the usual assumption of distinct training inputs and evaluating training error with each point included among its own neighbors. Then 1-NN has zero training error. kNN boundaries are generally nonlinear; 1-NN need not generalize better than 5-NN. This brute-force classifier compares each query with the training set, so prediction cost grows with its size. Strictly, conflicting labels at identical training inputs and distance tie-breaking can invalidate the unconditional wording of statement 2; without the usual assumption, only 4 is guaranteed for this implementation.

```
In [1]: # Local PyCharm / Jupyter setup
from local_setup import prepare
prepare()
```

Project: E:\DeepLearning\111\assignment1_colab\assignment1
Python: D:\Wei\python_3.12\python.exe
CIFAR-10 ready.

Softmax Classifier exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

In this exercise you will:

- implement a fully-vectorized **loss function** for the Softmax classifier.
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

```
In [2]: # Run some setup code for this notebook.
import random
import numpy as np
np.random.seed(0)
random.seed(0)
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

CIFAR-10 Data Loading and Preprocessing

```
In [3]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause memory issue)
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
```

```
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

E:\Deeplearning\111\assignment1_colab\assignment1\cs231n\data_utils.py:16: VisibleDeprecationWarning: dtype() align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)

```
return pickle.load(f, encoding="latin1")
```

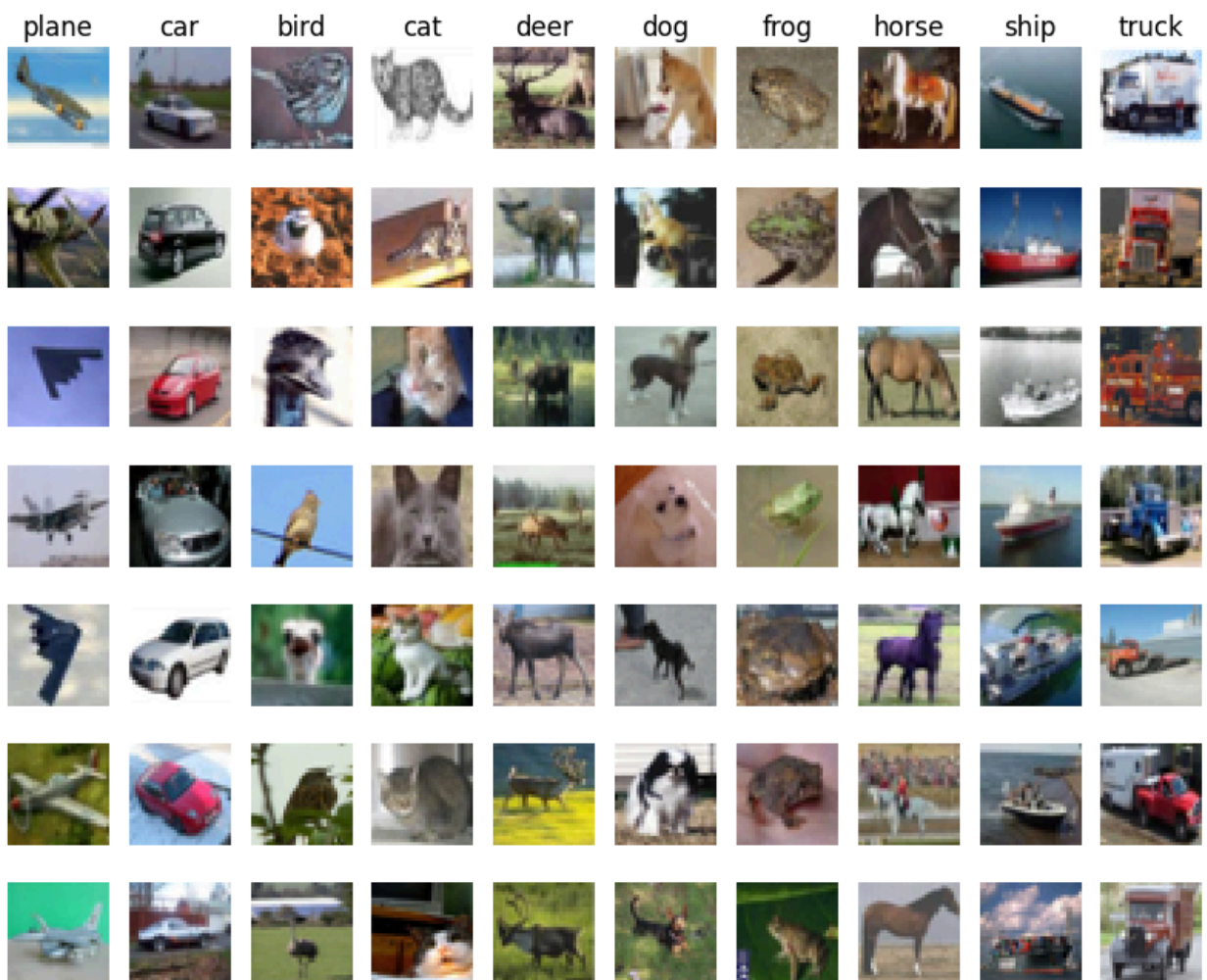
Training data shape: (50000, 32, 32, 3)

Training labels shape: (50000,)

Test data shape: (10000, 32, 32, 3)

Test labels shape: (10000,)

```
In [4]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



```
In [5]: # Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
```

```

num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)

```

```

Train data shape: (49000, 32, 32, 3)
Train labels shape: (49000,)
Validation data shape: (1000, 32, 32, 3)
Validation labels shape: (1000,)
Test data shape: (1000, 32, 32, 3)
Test labels shape: (1000,)

```

```

In [6]: # Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)

```

```

Training data shape: (49000, 3072)
Validation data shape: (1000, 3072)
Test data shape: (1000, 3072)
dev data shape: (500, 3072)

```

```

In [7]: # Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean image
plt.show()

# second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image

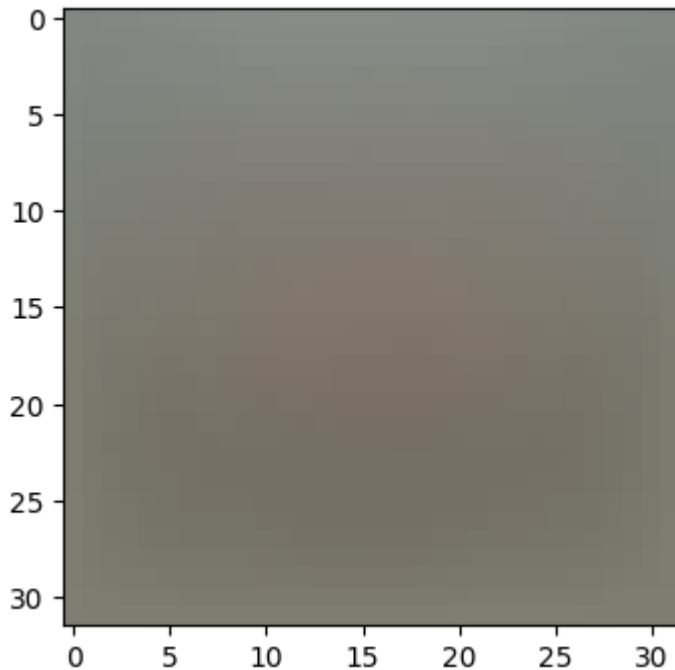
# third: append the bias dimension of ones (i.e. bias trick) so that our classifier
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])

```

```
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)
```

```
[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```



```
(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)
```

Softmax Classifier

Your code for this section will all be written inside `cs231n/classifiers/softmax.py`.

As you can see, we have prefilled the function `softmax_loss_naive` which uses for loops to evaluate the softmax loss function.

```
In [8]: # Evaluate the naive implementation of the loss we provided for you:
from cs231n.classifiers.softmax import softmax_loss_naive
import time

# generate a random Softmax classifier weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

```
loss: 2.407783
loss: 2.407783
sanity check: 2.302585
```

Inline Question 1

Why do we expect our loss to be close to $-\log(0.1)$? Explain briefly.**

Answer: There are ten classes. With small initial weights, scores are close to zero and the predicted probabilities are approximately uniform, so the correct class receives probability $1/10$. Its negative log probability is approximately $-\log(0.1) = 2.3026$; the

initial regularization contribution is small.

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the softmax loss function and implement it inline inside the function `softmax_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

```
In [9]: # Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions, and
# compare them with your analytically computed gradient. The numbers should match
# almost exactly along all dimensions.
from cs231n.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)

numerical: 1.363319 analytic: 1.363319, relative error: 3.210644e-08
numerical: 5.093382 analytic: 5.093382, relative error: 5.192849e-09
numerical: -0.458359 analytic: -0.458359, relative error: 4.808336e-08
numerical: 3.081793 analytic: 3.081793, relative error: 8.975605e-09
numerical: -0.936115 analytic: -0.936116, relative error: 3.205204e-08
numerical: 1.016566 analytic: 1.016566, relative error: 3.505579e-09
numerical: -0.545912 analytic: -0.545912, relative error: 7.066082e-09
numerical: -1.479715 analytic: -1.479715, relative error: 1.812468e-08
numerical: -4.028056 analytic: -4.028056, relative error: 4.841727e-10
numerical: -2.283214 analytic: -2.283214, relative error: 2.764453e-09
numerical: 0.186145 analytic: 0.186145, relative error: 3.889198e-07
numerical: 2.527135 analytic: 2.527135, relative error: 6.876509e-09
numerical: 1.625419 analytic: 1.625419, relative error: 5.346195e-08
numerical: -0.592100 analytic: -0.592100, relative error: 8.856139e-08
numerical: -1.134645 analytic: -1.134646, relative error: 1.643452e-08
numerical: 0.693054 analytic: 0.693054, relative error: 4.672303e-08
numerical: -1.403523 analytic: -1.403523, relative error: 4.298378e-08
numerical: -0.399118 analytic: -0.399118, relative error: 4.817106e-09
numerical: -0.583742 analytic: -0.583742, relative error: 5.612455e-08
numerical: -2.282394 analytic: -2.282394, relative error: 8.260199e-09
```

Inline Question 2

Although gradcheck is reliable softmax loss, it is possible that for SVM loss, once in a while, a dimension in the gradcheck will not match exactly. What could such a discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a svm loss gradient check could fail? How would change the margin affect of the frequency of this happening?

Note that SVM loss for a sample (x_i, y_i) is defined as:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

where j iterates over all classes except the correct class y_i and s_j denotes the classifier score for j^{th} class. Δ is a scalar margin. For more information, refer to 'Multiclass Support Vector Machine loss' on [this](#) page.

Hint: the SVM loss function is not strictly speaking differentiable.

Answer: The hinge loss has a kink at zero margin violation. A central finite difference can cross that kink, while the analytic implementation chooses one branch (or a subgradient at the kink). For $f(t)=\max(0,t)$, the central difference at $t=0$ is $1/2$, whereas an implementation may return 0. An isolated discrepancy near a kink need not indicate a bug; persistent discrepancies away from kinks do. Changing Delta moves the kink relative to the score differences, changing how often the finite-difference interval crosses it. There is no universal monotonic relationship between Delta and failure frequency; it depends on the score distribution and finite-difference step.

```
In [10]: # Next implement the function softmax_loss_vectorized; for now only compute the loss;
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

from cs231n.classifiers.softmax import softmax_loss_vectorized
tic = time.time()
loss_vectorized, _ = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# The losses should match but your vectorized implementation should be much faster.
print('difference: %f' % (loss_naive - loss_vectorized))
```

```
Naive loss: 2.407783e+00 computed in 0.033023s
Vectorized loss: 2.407783e+00 computed in 0.009252s
difference: 0.000000
```

```
In [11]: # Complete the implementation of softmax_loss_vectorized, and compute the gradient
# of the loss function in a vectorized way.

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)
```

```
Naive loss and gradient: computed in 0.039267s
Vectorized loss and gradient: computed in 0.008948s
difference: 0.000000
```

Stochastic Gradient Descent

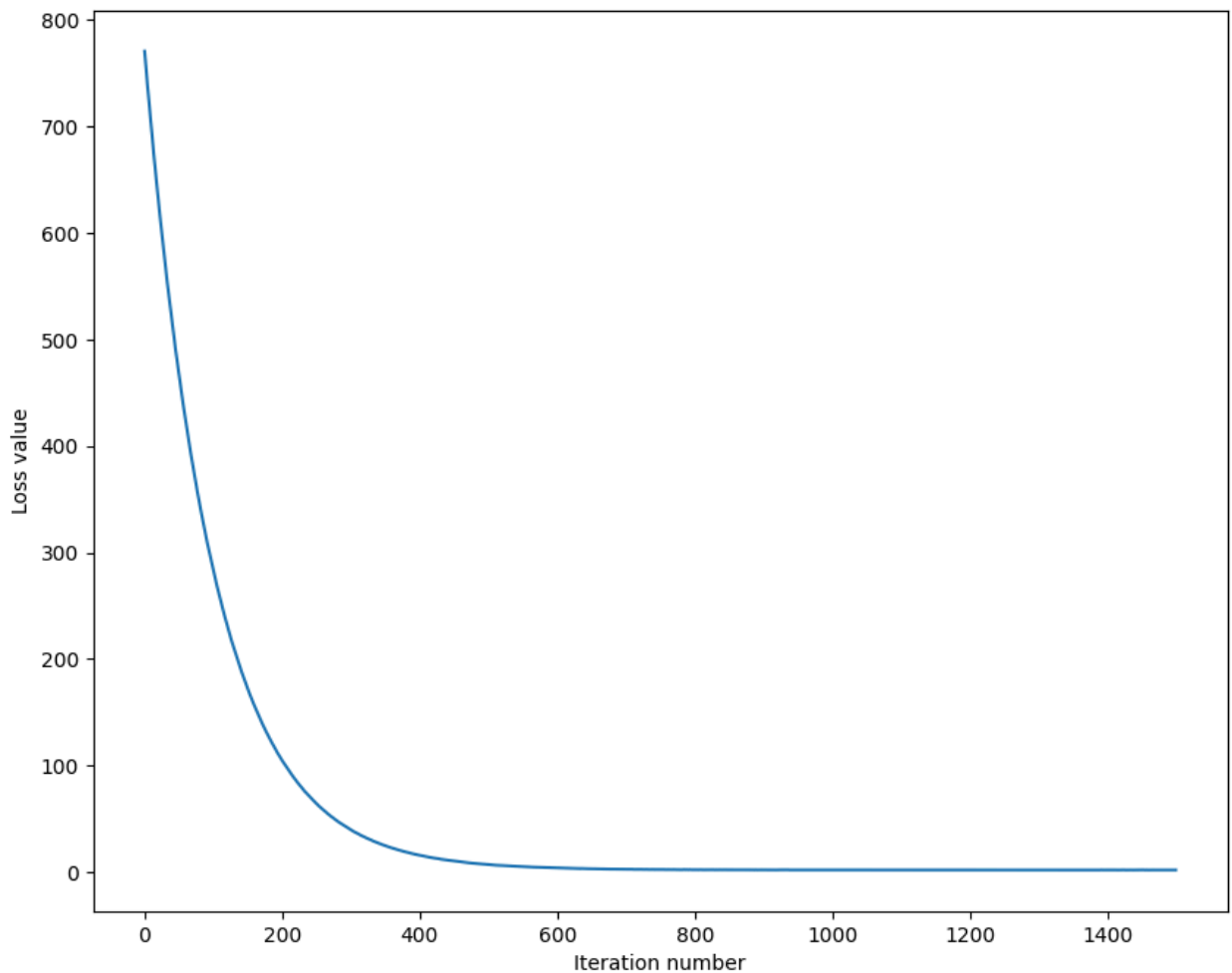
We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss. Your code for this part will be written inside

cs231n/classifiers/linear_classifier.py .

```
In [12]: # In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from cs231n.classifiers import import Softmax
softmax = Softmax()
tic = time.time()
loss_hist = softmax.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                          num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))
```

```
iteration 0 / 1500: loss 770.310559
iteration 100 / 1500: loss 282.774326
iteration 200 / 1500: loss 104.686952
iteration 300 / 1500: loss 39.714499
iteration 400 / 1500: loss 15.803542
iteration 500 / 1500: loss 7.121027
iteration 600 / 1500: loss 3.958640
iteration 700 / 1500: loss 2.734416
iteration 800 / 1500: loss 2.359490
iteration 900 / 1500: loss 2.144285
iteration 1000 / 1500: loss 2.123573
iteration 1100 / 1500: loss 2.133704
iteration 1200 / 1500: loss 2.052479
iteration 1300 / 1500: loss 2.024528
iteration 1400 / 1500: loss 2.077866
That took 3.072418s
```

```
In [13]: # A useful debugging strategy is to plot the loss as a function of
# iteration number:
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



```
In [14]: # Write the LinearClassifier.predict function and evaluate the performance on
# both the training and validation set
# You should get validation accuracy of about 0.34 (> 0.33).
y_train_pred = softmax.predict(X_train)
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = softmax.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

```
training accuracy: 0.327694
validation accuracy: 0.334000
```

```
In [15]: # Save the trained model for autograder.
softmax.save("softmax.npy")
```

softmax.npy saved.

```
In [16]: # Select hyperparameters using validation accuracy only.
learning_rates = [1e-7, 3e-7, 1e-6]
regularization_strengths = [1e3, 5e3, 1e4, 2.5e4]
results = {}
best_val = -1
best_softmax = None
for lr in learning_rates:
    for reg in regularization_strengths:
        np.random.seed(42)
        candidate = Softmax()
        candidate.train(X_train, y_train, learning_rate=lr, reg=reg,
                        num_iters=2000, batch_size=200)
        train_acc = np.mean(candidate.predict(X_train) == y_train)
        val_acc = np.mean(candidate.predict(X_val) == y_val)
        results[(lr, reg)] = (train_acc, val_acc)
        print('lr %e reg %e train accuracy: %f val accuracy: %f' %
              (lr, reg, train_acc, val_acc), flush=True)
        if val_acc > best_val:
            best_val = val_acc
            best_softmax = candidate
```

```

print('best validation accuracy achieved: %f' % best_val)
assert best_val > 0.36, 'Further validation-set tuning is needed.'

lr 1.000000e-07 reg 1.000000e+03 train accuracy: 0.284735 val accuracy: 0.292000
lr 1.000000e-07 reg 5.000000e+03 train accuracy: 0.357939 val accuracy: 0.361000
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.355531 val accuracy: 0.371000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.327551 val accuracy: 0.346000
lr 3.000000e-07 reg 1.000000e+03 train accuracy: 0.375306 val accuracy: 0.381000
lr 3.000000e-07 reg 5.000000e+03 train accuracy: 0.372939 val accuracy: 0.388000
lr 3.000000e-07 reg 1.000000e+04 train accuracy: 0.354122 val accuracy: 0.370000
lr 3.000000e-07 reg 2.500000e+04 train accuracy: 0.323612 val accuracy: 0.340000
lr 1.000000e-06 reg 1.000000e+03 train accuracy: 0.399306 val accuracy: 0.412000
lr 1.000000e-06 reg 5.000000e+03 train accuracy: 0.366796 val accuracy: 0.389000
lr 1.000000e-06 reg 1.000000e+04 train accuracy: 0.347673 val accuracy: 0.364000
lr 1.000000e-06 reg 2.500000e+04 train accuracy: 0.317571 val accuracy: 0.334000
best validation accuracy achieved: 0.412000

```

```

In [17]: # Visualize the cross-validation results
import math
import pdb

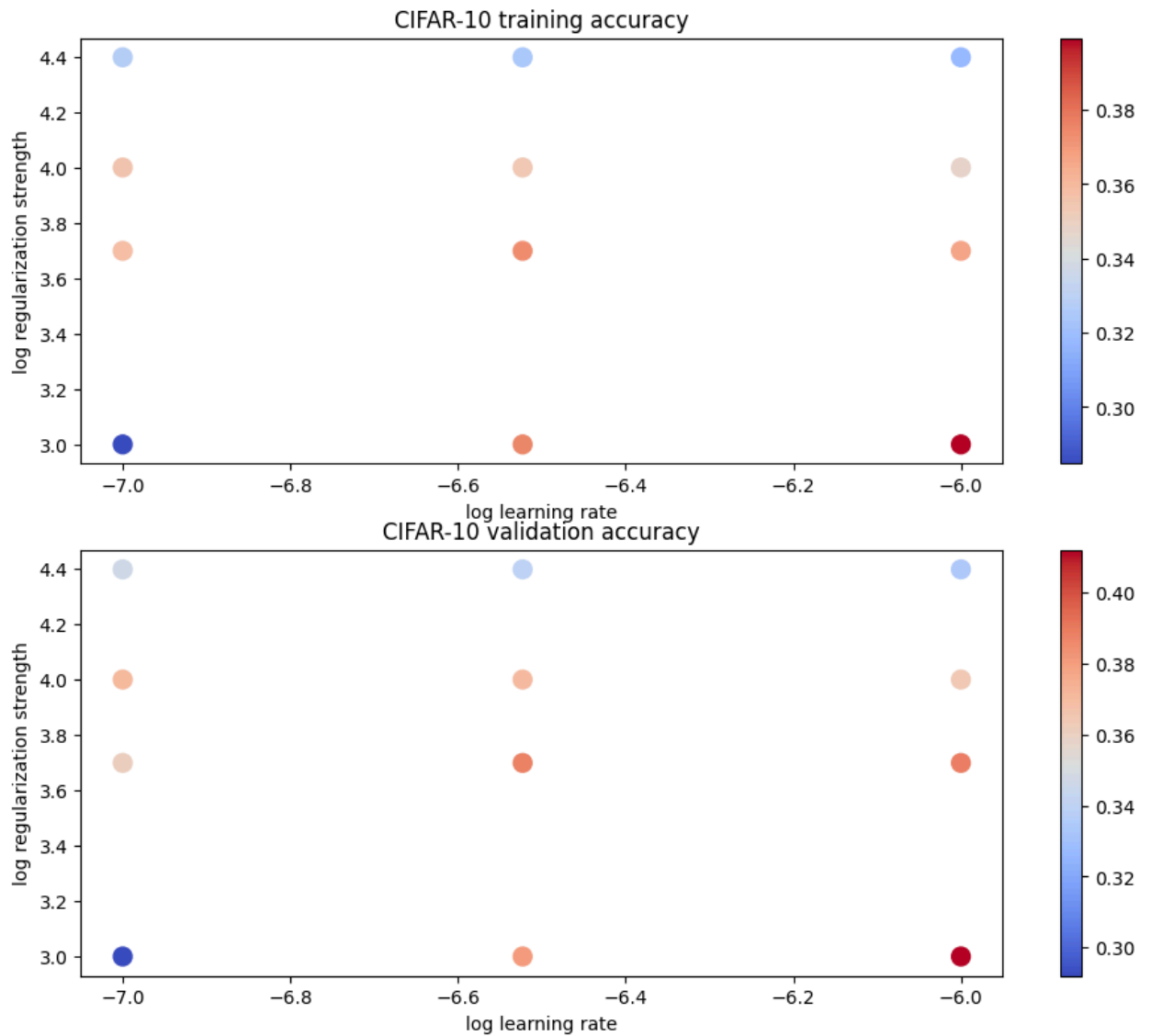
# pdb.set_trace()

x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.tight_layout(pad=3)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 validation accuracy')
plt.show()

```



```
In [18]: # Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax classifier on raw pixels final test set accuracy: %f' % test_accuracy)
```

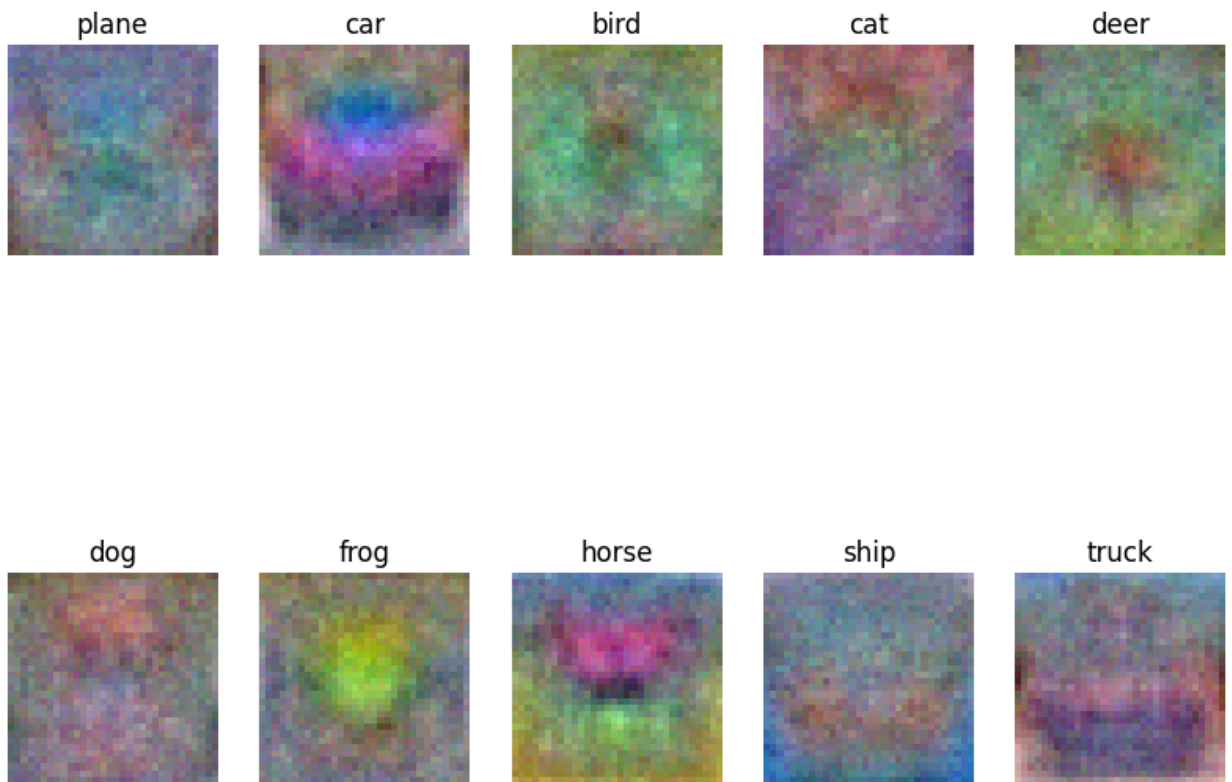
Softmax classifier on raw pixels final test set accuracy: 0.393000

```
In [19]: # Save best softmax model
best_softmax.save("best_softmax.npy")
```

best_softmax.npy saved.

```
In [20]: # Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, these may
# or may not be nice to look at.
w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])
```



Inline question 3

Describe what your visualized Softmax classifier weights look like, and offer a brief explanation for why they look the way they do.

Answer: The weights look like blurry, noisy color templates rather than sharp photographs. The plane and ship templates contain blue/cyan regions, bird and deer contain green regions, and the car/truck templates have darker lower regions. These patterns reflect discriminative color and location correlations learned from the training images. Each class has only one linear weight template, so different poses and appearances must share the same weights, producing broad, superimposed structure. The displayed colors are rescaled signed weights, not literal class-average pixel colors.

Inline Question 4 - True or False

Suppose the overall training loss is defined as the sum of the per-datapoint loss over all training examples. It is possible to add a new datapoint to a training set that would change the softmax loss, but leave the SVM loss unchanged.

Answer: True, for fixed weights and the stated sum of per-example losses. Add a point whose correct-class score exceeds every incorrect-class score by at least the SVM margin. Its hinge loss is zero, so the total SVM loss is unchanged. For finite scores, softmax assigns the correct class probability strictly below one, so this point contributes positive cross-entropy and increases the total softmax loss. The regularization term is unchanged at fixed weights.

In []:

```
In [1]: # Local PyCharm / Jupyter setup
        from local_setup import prepare
        prepare()
```

Project: E:\DeepLearning\111\assignment1_colab\assignment1
Python: D:\Wei\python_3.12\python.exe
CIFAR-10 ready.

Fully-Connected Neural Nets

In this exercise we will implement fully-connected networks using a modular approach. For each layer we will implement a `forward` and a `backward` function. The `forward` function will receive inputs, weights, and other parameters and will return both an output and a `cache` object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
    """ Receive inputs x and weights w """
    # Do some computations ...
    z = # ... some intermediate value
    # Do some more computations ...
    out = # the output

    cache = (x, w, z, out) # Values we need to compute gradients

    return out, cache
```

The backward pass will receive upstream derivatives and the `cache` object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
    """
    Receive dout (derivative of loss with respect to outputs) and cache,
    and compute derivative with respect to inputs.
    """
    # Unpack cache values
    x, w, z, out = cache

    # Use values in cache to compute derivatives
    dx = # Derivative of loss with respect to x
    dw = # Derivative of loss with respect to w

    return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

```
In [2]: # As usual, a bit of setup
        from __future__ import print_function
        import time
        import numpy as np
        import matplotlib.pyplot as plt
        from cs231n.classifiers.fc_net import *
        from cs231n.data_utils import get_CIFAR10_data
        from cs231n.gradient_check import eval_numerical_gradient, eval_numerical_gradient_array
        from cs231n.solver import Solver

        %matplotlib inline
        plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
        plt.rcParams['image.interpolation'] = 'nearest'
        plt.rcParams['image.cmap'] = 'gray'

        # for auto-reloading external modules
        # see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
        %load_ext autoreload
        %autoreload 2
```

```
def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

In [3]: # Load the (preprocessed) CIFAR10 data.

```
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print('%s: ' % k, v.shape)
```

```
E:\Deeplearning\111\assignment1_colab\assignment1\cs231n\data_utils.py:16: VisibleDeprecationWarning: dtype
pe(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple t
o create a subarray type? (Deprecated NumPy 2.4)
    return pickle.load(f, encoding="latin1")
('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))
```

Affine layer: forward

Open the file `cs231n/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementaion by running the following:

In [4]: # Test the affine_forward function

```
num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape), output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,  3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference: 9.769848888397517e-10
```

Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

In [5]: # Test the affine_backward function

```
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w, dout)
```

```

db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b, dout)

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))

```

Testing affine_backward function:
dx error: 5.399100368651805e-11
dw error: 9.904211865398145e-11
db error: 2.4122867568119087e-11

ReLU activation: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function and test your implementation using the following:

```

In [6]: # Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.],
                        [ 0.,          0.,          0.04545455, 0.13636364],
                        [ 0.22727273, 0.31818182, 0.40909091, 0.5]])

# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))

```

Testing relu_forward function:
difference: 4.999999798022158e-08

ReLU activation: backward

Now implement the backward pass for the ReLU activation function in the `relu_backward` function and test your implementation using numeric gradient checking:

```

In [7]: np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))

```

Testing relu_backward function:
dx error: 3.2756349136310288e-12

Inline Question 1:

We've only asked you to implement ReLU, but there are a number of different activation functions that one could use in neural networks, each with its pros and cons. In particular, an issue commonly seen with activation functions is getting zero (or close to zero) gradient flow during backpropagation. Which of the following activation functions have this problem? If you consider these functions in the one dimensional case, what types of input would lead to this behaviour?

1. Sigmoid
2. ReLU
3. Leaky ReLU

Answer: Sigmoid and ReLU have this problem. Sigmoid has a derivative close to zero for inputs with large positive or negative magnitude because it saturates. ReLU has zero derivative for negative inputs (we also choose zero at the nondifferentiable point $x=0$). Leaky ReLU with a fixed positive negative-side slope preserves a nonzero gradient on negative inputs; its gradient can still be small if that slope is chosen extremely small.

"Sandwich" layers

There are some common patterns of layers that are frequently used in neural nets. For example, affine layers are frequently followed by a ReLU nonlinearity. To make these common patterns easy, we define several convenience layers in the file

`cs231n/layer_utils.py` .

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

```
In [8]: from cs231n.layer_utils import affine_relu_forward, affine_relu_backward
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w, b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w, b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w, b)[0], b, dout)

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_relu_forward and affine_relu_backward:
dx error:  2.299579177309368e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12
```

Loss layers: Softmax

Now implement the loss and gradient for softmax in the `softmax_loss` function in `cs231n/layers.py`. These should be similar to what you implemented in `cs231n/classifiers/softmax.py`. Other loss functions (e.g. `svm_loss`) can also be implemented in a modular way, however, it is not required for this assignment.

You can make sure that the implementations are correct by running the following:

```
In [9]: np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)

dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x, verbose=False)
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should be around e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing softmax_loss:
loss: 2.302545844500738
dx error: 9.384673161989355e-09
```

Two-layer network

Open the file `cs231n/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. Read through it to make sure you understand the API. You can run the cell below to test your implementation.

```
In [10]: np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
w1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
w2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert w1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert w2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
    [[11.53165108, 12.2917344, 13.05181771, 13.81190102, 14.57198434, 15.33206765, 16.09215096],
     [12.05769098, 12.74614105, 13.43459113, 14.1230412, 14.81149128, 15.49994135, 16.18839143],
     [12.58373087, 13.20054771, 13.81736455, 14.43418138, 15.05099822, 15.66781506, 16.2846319 ]])
```

```

scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
    print('Running numeric gradient check with reg = ', reg)
    model.reg = reg
    loss, grads = model.loss(X, y)

    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
        print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))

```

```

Testing initialization ...
Testing test-time forward pass ...
Testing training loss (no regularization)
Running numeric gradient check with reg = 0.0
W1 relative error: 1.52e-08
W2 relative error: 3.21e-10
b1 relative error: 8.37e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg = 0.7
W1 relative error: 2.53e-07
W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 7.76e-10

```

Solver

Open the file `cs231n/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves about 36% accuracy on the validation set.

```

In [11]: np.random.seed(0)
input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
model = TwoLayerNet(input_size, hidden_size, num_classes)
solver = Solver(model, data, update_rule='sgd',
                optim_config={'learning_rate': 1e-3}, lr_decay=0.95,
                num_epochs=10, batch_size=100, print_every=500)
solver.train()

```

```

(Iteration 1 / 4900) loss: 2.303478
(Epoch 0 / 10) train acc: 0.111000; val_acc: 0.132000
(Epoch 1 / 10) train acc: 0.447000; val_acc: 0.448000
(Iteration 501 / 4900) loss: 1.563530
(Epoch 2 / 10) train acc: 0.490000; val_acc: 0.462000
(Iteration 1001 / 4900) loss: 1.337231
(Epoch 3 / 10) train acc: 0.487000; val_acc: 0.442000
(Iteration 1501 / 4900) loss: 1.344426
(Epoch 4 / 10) train acc: 0.515000; val_acc: 0.484000
(Iteration 2001 / 4900) loss: 1.245396
(Epoch 5 / 10) train acc: 0.544000; val_acc: 0.481000
(Iteration 2501 / 4900) loss: 1.278434
(Epoch 6 / 10) train acc: 0.507000; val_acc: 0.473000
(Iteration 3001 / 4900) loss: 1.131368

```

```
(Epoch 7 / 10) train acc: 0.546000; val_acc: 0.485000  
(Iteration 3501 / 4900) loss: 1.314299  
(Epoch 8 / 10) train acc: 0.571000; val_acc: 0.490000  
(Iteration 4001 / 4900) loss: 1.213247  
(Epoch 9 / 10) train acc: 0.534000; val_acc: 0.509000  
(Iteration 4501 / 4900) loss: 1.312731  
(Epoch 10 / 10) train acc: 0.590000; val_acc: 0.505000
```

Debug the training

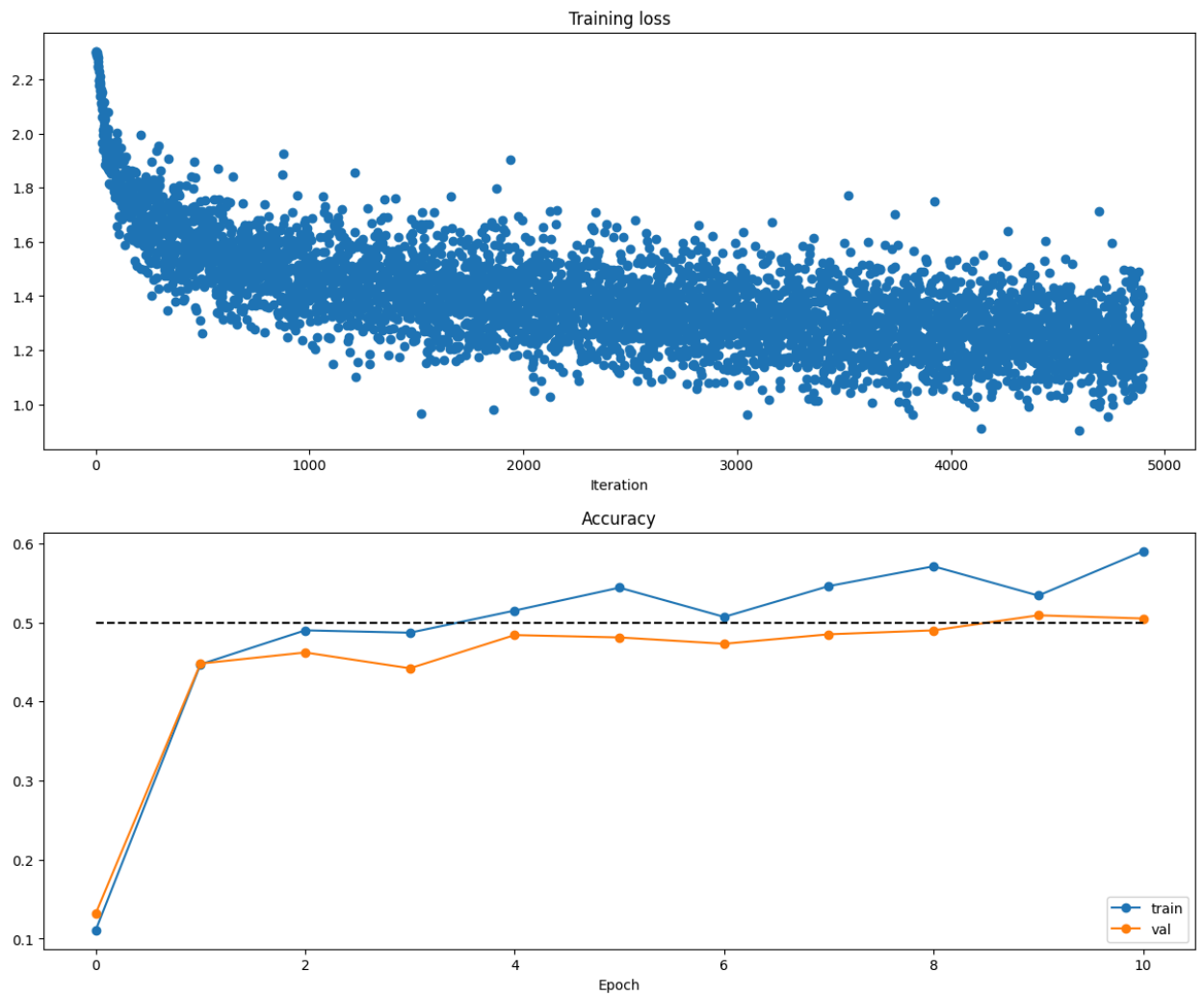
With the default parameters we provided above, you should get a validation accuracy of about 0.36 on the validation set. This isn't very good.

One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

In [12]: *# Run this cell to visualize training loss and train / val accuracy*

```
plt.subplot(2, 1, 1)  
plt.title('Training loss')  
plt.plot(solver.loss_history, 'o')  
plt.xlabel('Iteration')  
  
plt.subplot(2, 1, 2)  
plt.title('Accuracy')  
plt.plot(solver.train_acc_history, '-o', label='train')  
plt.plot(solver.val_acc_history, '-o', label='val')  
plt.plot([0.5] * len(solver.val_acc_history), 'k--')  
plt.xlabel('Epoch')  
plt.legend(loc='lower right')  
plt.gcf().set_size_inches(15, 12)  
plt.show()
```

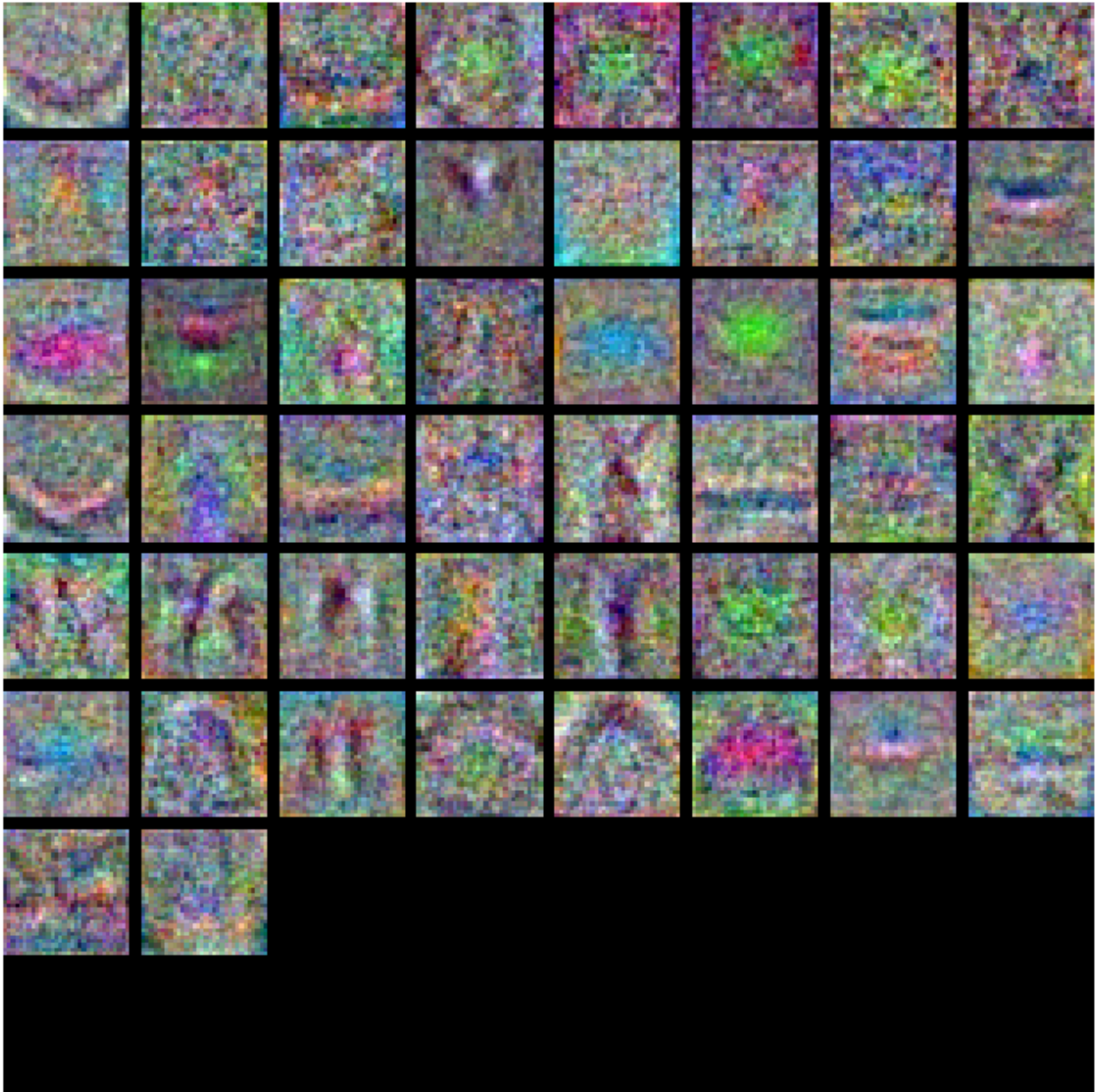


```
In [13]: from cs231n.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```



Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, number of training epochs, and regularization strength. You might also consider tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: Your goal in this exercise is to get as good of a result on CIFAR-10 as you can (52% could serve as a reference), with a fully-connected Neural Network. Feel free to implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

```
In [14]: # Search only on training and validation data; Solver restores the best epoch.
configs = [(128, 1e-3, 0.1), (256, 1e-3, 0.1), (256, 5e-4, 0.05)]
best_model = None
best_val = -1
results = {}
for hidden, lr, reg in configs:
    np.random.seed(42)
    candidate = TwoLayerNet(hidden_dim=hidden, weight_scale=1e-2, reg=reg)
    candidate_solver = Solver(candidate, data, update_rule='sgd',
                              optim_config={'learning_rate': lr}, lr_decay=0.95,
                              num_epochs=20, batch_size=200, print_every=1000)
    candidate_solver.train()
    val_acc = candidate_solver.check_accuracy(data['X_val'], data['y_val'])
    results[(hidden, lr, reg)] = val_acc
    print('hidden=%d lr=%g reg=%g validation=%f' % (hidden, lr, reg, val_acc))
    if val_acc > best_val:
        best_val = val_acc
        best_model = candidate
        best_solver = candidate_solver
print('Best validation accuracy:', best_val)
assert best_val > 0.48, 'More validation-set tuning is needed.'
```

```
(Iteration 1 / 4900) loss: 6.352124
(Epoch 0 / 20) train acc: 0.133000; val_acc: 0.135000
(Epoch 1 / 20) train acc: 0.388000; val_acc: 0.394000
(Epoch 2 / 20) train acc: 0.437000; val_acc: 0.423000
(Epoch 3 / 20) train acc: 0.460000; val_acc: 0.428000
(Epoch 4 / 20) train acc: 0.499000; val_acc: 0.460000
(Iteration 1001 / 4900) loss: 3.132110
(Epoch 5 / 20) train acc: 0.480000; val_acc: 0.473000
(Epoch 6 / 20) train acc: 0.501000; val_acc: 0.477000
(Epoch 7 / 20) train acc: 0.521000; val_acc: 0.464000
(Epoch 8 / 20) train acc: 0.526000; val_acc: 0.509000
(Iteration 2001 / 4900) loss: 2.621609
(Epoch 9 / 20) train acc: 0.554000; val_acc: 0.475000
(Epoch 10 / 20) train acc: 0.572000; val_acc: 0.477000
(Epoch 11 / 20) train acc: 0.571000; val_acc: 0.499000
(Epoch 12 / 20) train acc: 0.562000; val_acc: 0.497000
(Iteration 3001 / 4900) loss: 2.438925
(Epoch 13 / 20) train acc: 0.596000; val_acc: 0.498000
(Epoch 14 / 20) train acc: 0.578000; val_acc: 0.493000
(Epoch 15 / 20) train acc: 0.587000; val_acc: 0.492000
(Epoch 16 / 20) train acc: 0.606000; val_acc: 0.480000
(Iteration 4001 / 4900) loss: 2.295400
(Epoch 17 / 20) train acc: 0.596000; val_acc: 0.491000
(Epoch 18 / 20) train acc: 0.597000; val_acc: 0.511000
(Epoch 19 / 20) train acc: 0.589000; val_acc: 0.509000
(Epoch 20 / 20) train acc: 0.609000; val_acc: 0.500000
hidden=128 lr=0.001 reg=0.1 validation=0.511000
(Iteration 1 / 4900) loss: 10.903026
(Epoch 0 / 20) train acc: 0.141000; val_acc: 0.148000
(Epoch 1 / 20) train acc: 0.402000; val_acc: 0.383000
(Epoch 2 / 20) train acc: 0.456000; val_acc: 0.453000
(Epoch 3 / 20) train acc: 0.512000; val_acc: 0.452000
(Epoch 4 / 20) train acc: 0.493000; val_acc: 0.453000
(Iteration 1001 / 4900) loss: 4.546364
(Epoch 5 / 20) train acc: 0.529000; val_acc: 0.491000
(Epoch 6 / 20) train acc: 0.517000; val_acc: 0.493000
(Epoch 7 / 20) train acc: 0.570000; val_acc: 0.490000
(Epoch 8 / 20) train acc: 0.573000; val_acc: 0.519000
(Iteration 2001 / 4900) loss: 4.033099
```

```

(Epoch 9 / 20) train acc: 0.598000; val_acc: 0.499000
(Epoch 10 / 20) train acc: 0.600000; val_acc: 0.508000
(Epoch 11 / 20) train acc: 0.633000; val_acc: 0.489000
(Epoch 12 / 20) train acc: 0.604000; val_acc: 0.487000
(Iteration 3001 / 4900) loss: 3.667219
(Epoch 13 / 20) train acc: 0.606000; val_acc: 0.518000
(Epoch 14 / 20) train acc: 0.632000; val_acc: 0.509000
(Epoch 15 / 20) train acc: 0.640000; val_acc: 0.496000
(Epoch 16 / 20) train acc: 0.619000; val_acc: 0.503000
(Iteration 4001 / 4900) loss: 3.312200
(Epoch 17 / 20) train acc: 0.628000; val_acc: 0.512000
(Epoch 18 / 20) train acc: 0.691000; val_acc: 0.521000
(Epoch 19 / 20) train acc: 0.673000; val_acc: 0.511000
(Epoch 20 / 20) train acc: 0.671000; val_acc: 0.539000
hidden=256 lr=0.001 reg=0.1 validation=0.539000
(Iteration 1 / 4900) loss: 8.929160
(Epoch 0 / 20) train acc: 0.136000; val_acc: 0.136000
(Epoch 1 / 20) train acc: 0.412000; val_acc: 0.368000
(Epoch 2 / 20) train acc: 0.463000; val_acc: 0.423000
(Epoch 3 / 20) train acc: 0.512000; val_acc: 0.451000
(Epoch 4 / 20) train acc: 0.472000; val_acc: 0.443000
(Iteration 1001 / 4900) loss: 3.243797
(Epoch 5 / 20) train acc: 0.490000; val_acc: 0.460000
(Epoch 6 / 20) train acc: 0.490000; val_acc: 0.474000
(Epoch 7 / 20) train acc: 0.502000; val_acc: 0.482000
(Epoch 8 / 20) train acc: 0.533000; val_acc: 0.485000
(Iteration 2001 / 4900) loss: 3.172451
(Epoch 9 / 20) train acc: 0.517000; val_acc: 0.472000
(Epoch 10 / 20) train acc: 0.529000; val_acc: 0.486000
(Epoch 11 / 20) train acc: 0.570000; val_acc: 0.479000
(Epoch 12 / 20) train acc: 0.529000; val_acc: 0.487000
(Iteration 3001 / 4900) loss: 3.064842
(Epoch 13 / 20) train acc: 0.548000; val_acc: 0.503000
(Epoch 14 / 20) train acc: 0.553000; val_acc: 0.487000
(Epoch 15 / 20) train acc: 0.583000; val_acc: 0.501000
(Epoch 16 / 20) train acc: 0.566000; val_acc: 0.497000
(Iteration 4001 / 4900) loss: 3.011888
(Epoch 17 / 20) train acc: 0.569000; val_acc: 0.492000
(Epoch 18 / 20) train acc: 0.596000; val_acc: 0.497000
(Epoch 19 / 20) train acc: 0.577000; val_acc: 0.506000
(Epoch 20 / 20) train acc: 0.588000; val_acc: 0.518000
hidden=256 lr=0.0005 reg=0.05 validation=0.518000
Best validation accuracy: 0.539

```

Test your model!

Run your best model on the validation and test sets. You should achieve above 48% accuracy on the validation set and the test set.

```

In [15]: y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
         print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())

```

Validation set accuracy: 0.539

```

In [16]: y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
         print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())

```

Test set accuracy: 0.516

```

In [17]: # Save best model
         best_model.save("best_two_layer_net.npy")

```

best_two_layer_net.npy saved.

Inline Question 2:

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

Answer: Choices 1 and 3. More representative training data can reduce overfitting, and stronger regularization can discourage overly complex fits. Adding hidden units increases capacity and does not generally reduce the training-test gap. A smaller gap alone does not guarantee better test accuracy: excessive regularization can cause underfitting.

In []:

```
In [1]: # Local PyCharm / Jupyter setup
from local_setup import prepare
prepare()
```

Project: E:\DeepLearning\111\assignment1_colab\assignment1
Python: D:\Wei\python_3.12\python.exe
CIFAR-10 ready.

Image features exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

```
In [2]: import random
import numpy as np
np.random.seed(0)
random.seed(0)
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

```
In [3]: from cs231n.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000):
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

    # Cleaning up variables to prevent loading data multiple times (which may cause memory issue)
    try:
        del X_train, y_train
        del X_test, y_test
        print('Clear previously loaded data.')
    except:
        pass

    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
```

```

X_val = X_train[mask]
y_val = y_train[mask]
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

return X_train, y_train, X_val, y_val, X_test, y_test

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()

```

E:\Deeplearning\111\assignment1_colab\assignment1\cs231n\data_utils.py:16: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)

```

return pickle.load(f, encoding="latin1")

```

Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The `extract_features` function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

```

In [4]: from cs231n.features import *

# num_color_bins = 10 # Number of bins in the color histogram
num_color_bins = 25 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img, nbin=num_color_bins)]
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
std_feat = np.maximum(std_feat, 1e-12)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])

```

```
assert all(np.isfinite(a).all() for a in (X_train_feats, X_val_feats, X_test_feats))
print("Feature matrix shapes:", X_train_feats.shape, X_val_feats.shape, X_test_feats.shape)
```

```
Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images
Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
Done extracting features for 46000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images
Done extracting features for 49000 / 49000 images
Feature matrix shapes: (49000, 170) (1000, 170) (1000, 170)
```

Train Softmax classifier on features

Using the Softmax code developed earlier in the assignment, train Softmax classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

```
In [5]: from cs231n.classifiers.linear_classifier import Softmax
# Standardized features require different hyperparameters from raw pixels.
learning_rates = [0.03, 0.1, 0.3]
regularization_strengths = [0.001, 0.01]
results = {}
best_val = -1
best_softmax = None
for lr in learning_rates:
    for reg in regularization_strengths:
        np.random.seed(42)
```

```

candidate = Softmax()
candidate.train(X_train_feats, y_train, learning_rate=lr, reg=reg,
               num_iters=2000, batch_size=200)
train_acc = np.mean(candidate.predict(X_train_feats) == y_train)
val_acc = np.mean(candidate.predict(X_val_feats) == y_val)
results[(lr, reg)] = (train_acc, val_acc)
print('lr=%g reg=%g train=%f validation=%f' % (lr, reg, train_acc, val_acc))
if val_acc > best_val:
    best_val, best_softmax = val_acc, candidate
print('Best validation accuracy:', best_val)
assert best_val > 0.42

```

```

lr=0.03 reg=0.001 train=0.523980 validation=0.517000
lr=0.03 reg=0.01 train=0.520245 validation=0.510000
lr=0.1 reg=0.001 train=0.520592 validation=0.517000
lr=0.1 reg=0.01 train=0.515265 validation=0.502000
lr=0.3 reg=0.001 train=0.508633 validation=0.494000
lr=0.3 reg=0.01 train=0.500510 validation=0.492000
Best validation accuracy: 0.517

```

```

In [6]: # Evaluate your trained Softmax on the test set: you should be able to get at least 0.42
y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)

```

0.506

```

In [7]: # Save best softmax model
best_softmax.save("best_softmax_features.npy")

```

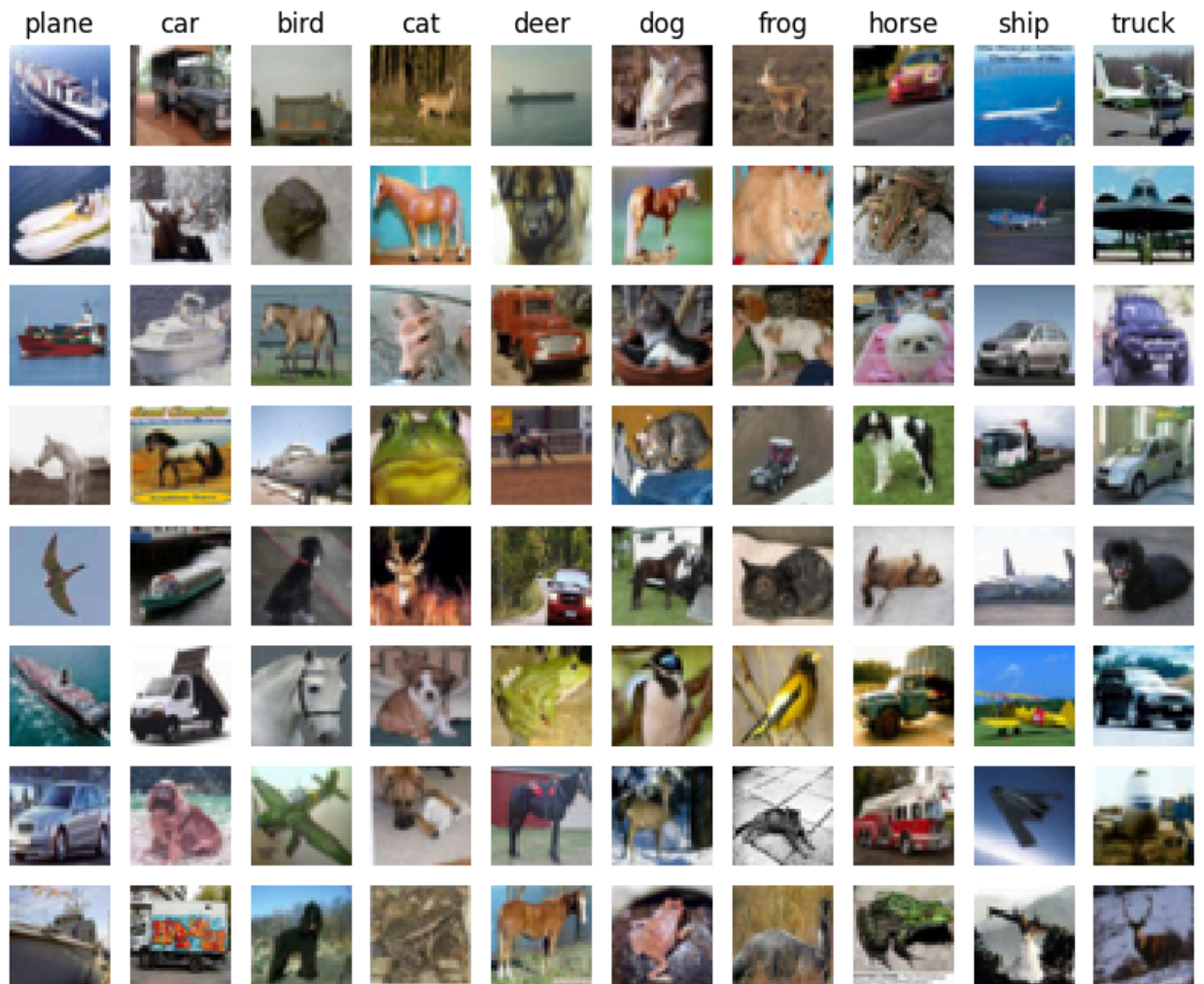
best_softmax_features.npy
saved.

```

In [8]: # An important way to gain intuition about how an algorithm works is to
# visualize the mistakes that it makes. In this visualization, we show examples
# of images that are misclassified by our current system. The first column
# shows images that our system labeled as "plane" but whose true label is
# something other than "plane".

examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    idxs = np.random.choice(idxs, min(examples_per_class, len(idxs)), replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls + 1)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)
plt.show()
# Summarize the most common errors to support the written analysis.
confusion = np.zeros((10, 10), dtype=int)
np.add.at(confusion, (y_test, y_test_pred), 1)
np.fill_diagonal(confusion, 0)
for flat_index in np.argsort(confusion.ravel())[-8:][::-1]:
    actual, predicted = np.unravel_index(flat_index, confusion.shape)
    print("%s -> %s: %d" % (classes[actual], classes[predicted], confusion[actual, predicted]))

```



Neural Network on image features

Earlier in this assignment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

```
In [9]: # Preserve the bias-augmented arrays for rerunning Softmax cells.
X_train_nn = X_train_feats[:, :-1]
X_val_nn = X_val_feats[:, :-1]
X_test_nn = X_test_feats[:, :-1]
print('Neural network feature shape:', X_train_nn.shape)
```

Neural network feature shape:
(49000, 169)

```
In [10]: from cs231n.classifiers.fc_net import TwoLayerNet
from cs231n.solver import Solver
input_dim = X_train_nn.shape[1]
data = {'X_train': X_train_nn, 'y_train': y_train,
        'X_val': X_val_nn, 'y_val': y_val,
        'X_test': X_test_nn, 'y_test': y_test}
configs = [(500, 0.1, 0.001), (500, 0.1, 0.01), (500, 0.05, 0.001)]
best_net = None
best_nn_val = -1
nn_results = {}
for hidden, lr, reg in configs:
    np.random.seed(42)
    net = TwoLayerNet(input_dim, hidden, 10, weight_scale=0.05, reg=reg)
    solver = Solver(net, data, update_rule='sgd',
                    optim_config={'learning_rate': lr}, lr_decay=0.97,
                    num_epochs=25, batch_size=200, print_every=1000)
    solver.train()
    val_acc = solver.check_accuracy(data['X_val'], data['y_val'])
    nn_results[(hidden, lr, reg)] = val_acc
    print('hidden=%d lr=%g reg=%g validation=%f' % (hidden, lr, reg, val_acc))
    if val_acc > best_nn_val:
        best_nn_val, best_net, best_solver = val_acc, net, solver
print('Best neural network validation accuracy:', best_nn_val)
plt.plot(best_solver.loss_history)
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()
plt.plot(best_solver.train_acc_history, label='train (sampled)')
plt.plot(best_solver.val_acc_history, label='validation')
plt.xlabel('Evaluation index')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

```
(Iteration 1 / 6125) loss: 2.492701
(Epoch 0 / 25) train acc: 0.107000; val_acc: 0.128000
(Epoch 1 / 25) train acc: 0.533000; val_acc: 0.518000
(Epoch 2 / 25) train acc: 0.566000; val_acc: 0.544000
(Epoch 3 / 25) train acc: 0.582000; val_acc: 0.556000
(Epoch 4 / 25) train acc: 0.576000; val_acc: 0.577000
(Iteration 1001 / 6125) loss: 1.176566
(Epoch 5 / 25) train acc: 0.616000; val_acc: 0.580000
(Epoch 6 / 25) train acc: 0.630000; val_acc: 0.574000
(Epoch 7 / 25) train acc: 0.615000; val_acc: 0.586000
```

```

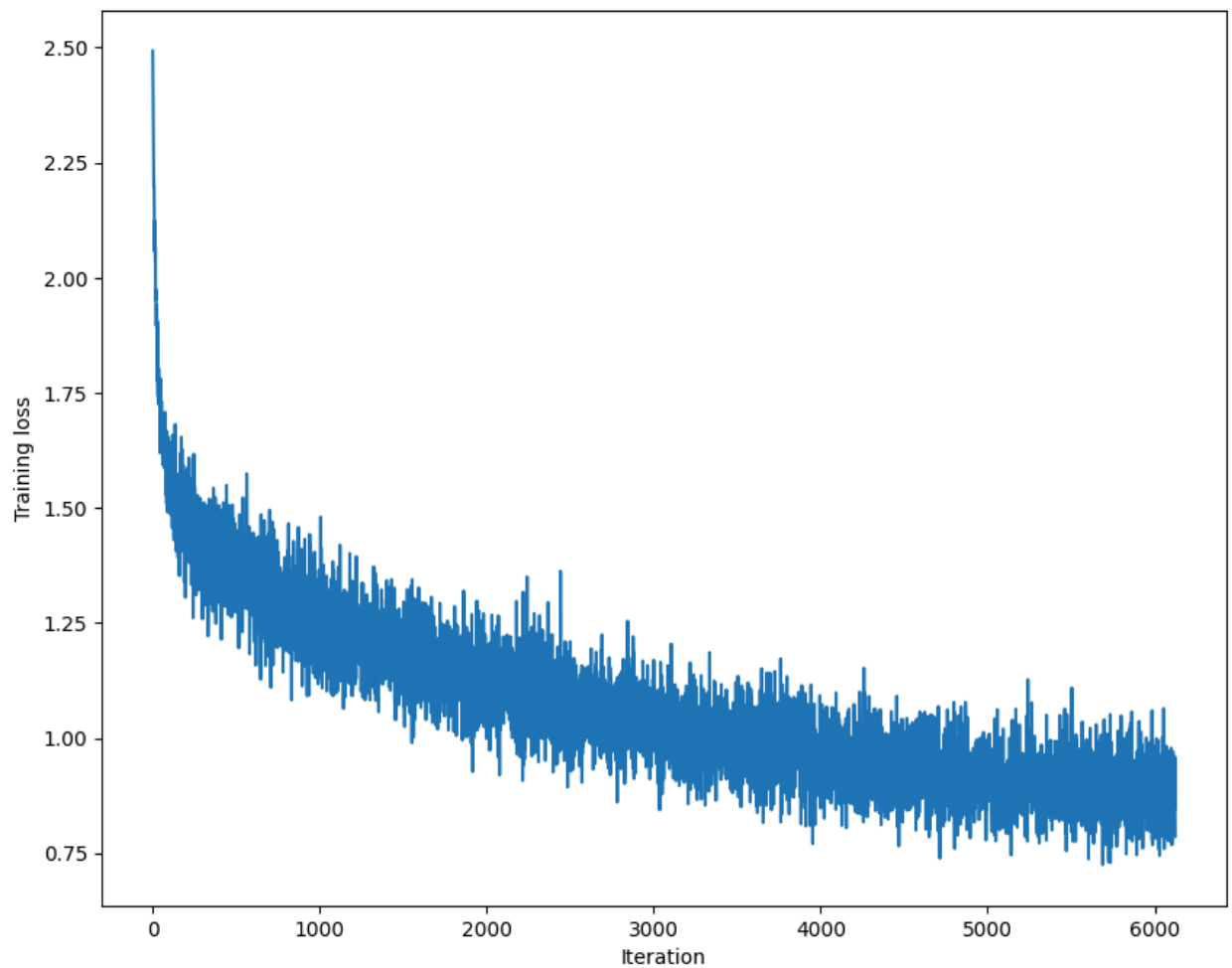
(Epoch 8 / 25) train acc: 0.666000; val_acc: 0.586000
(Iteration 2001 / 6125) loss: 1.076947
(Epoch 9 / 25) train acc: 0.639000; val_acc: 0.585000
(Epoch 10 / 25) train acc: 0.673000; val_acc: 0.586000
(Epoch 11 / 25) train acc: 0.670000; val_acc: 0.600000
(Epoch 12 / 25) train acc: 0.701000; val_acc: 0.614000
(Iteration 3001 / 6125) loss: 1.169338
(Epoch 13 / 25) train acc: 0.701000; val_acc: 0.594000
(Epoch 14 / 25) train acc: 0.743000; val_acc: 0.610000
(Epoch 15 / 25) train acc: 0.721000; val_acc: 0.601000
(Epoch 16 / 25) train acc: 0.724000; val_acc: 0.604000
(Iteration 4001 / 6125) loss: 0.847764
(Epoch 17 / 25) train acc: 0.733000; val_acc: 0.599000
(Epoch 18 / 25) train acc: 0.750000; val_acc: 0.612000
(Epoch 19 / 25) train acc: 0.728000; val_acc: 0.602000
(Epoch 20 / 25) train acc: 0.743000; val_acc: 0.594000
(Iteration 5001 / 6125) loss: 0.992228
(Epoch 21 / 25) train acc: 0.772000; val_acc: 0.599000
(Epoch 22 / 25) train acc: 0.746000; val_acc: 0.603000
(Epoch 23 / 25) train acc: 0.756000; val_acc: 0.606000
(Epoch 24 / 25) train acc: 0.774000; val_acc: 0.601000
(Iteration 6001 / 6125) loss: 0.899786
(Epoch 25 / 25) train acc: 0.777000; val_acc: 0.594000
hidden=500 lr=0.1 reg=0.001 validation=0.614000
(Iteration 1 / 6125) loss: 3.501797
(Epoch 0 / 25) train acc: 0.107000; val_acc: 0.128000
(Epoch 1 / 25) train acc: 0.531000; val_acc: 0.520000
(Epoch 2 / 25) train acc: 0.547000; val_acc: 0.541000
(Epoch 3 / 25) train acc: 0.551000; val_acc: 0.543000
(Epoch 4 / 25) train acc: 0.542000; val_acc: 0.549000
(Iteration 1001 / 6125) loss: 1.515140
(Epoch 5 / 25) train acc: 0.577000; val_acc: 0.544000
(Epoch 6 / 25) train acc: 0.590000; val_acc: 0.551000
(Epoch 7 / 25) train acc: 0.550000; val_acc: 0.554000
(Epoch 8 / 25) train acc: 0.601000; val_acc: 0.560000
(Iteration 2001 / 6125) loss: 1.382075
(Epoch 9 / 25) train acc: 0.570000; val_acc: 0.550000
(Epoch 10 / 25) train acc: 0.597000; val_acc: 0.575000
(Epoch 11 / 25) train acc: 0.577000; val_acc: 0.557000
(Epoch 12 / 25) train acc: 0.583000; val_acc: 0.567000
(Iteration 3001 / 6125) loss: 1.476737
(Epoch 13 / 25) train acc: 0.600000; val_acc: 0.567000
(Epoch 14 / 25) train acc: 0.615000; val_acc: 0.568000
(Epoch 15 / 25) train acc: 0.592000; val_acc: 0.570000
(Epoch 16 / 25) train acc: 0.587000; val_acc: 0.563000
(Iteration 4001 / 6125) loss: 1.301214
(Epoch 17 / 25) train acc: 0.598000; val_acc: 0.573000
(Epoch 18 / 25) train acc: 0.611000; val_acc: 0.581000
(Epoch 19 / 25) train acc: 0.602000; val_acc: 0.577000
(Epoch 20 / 25) train acc: 0.604000; val_acc: 0.565000
(Iteration 5001 / 6125) loss: 1.406046
(Epoch 21 / 25) train acc: 0.627000; val_acc: 0.574000
(Epoch 22 / 25) train acc: 0.582000; val_acc: 0.574000
(Epoch 23 / 25) train acc: 0.597000; val_acc: 0.575000
(Epoch 24 / 25) train acc: 0.597000; val_acc: 0.583000
(Iteration 6001 / 6125) loss: 1.373650
(Epoch 25 / 25) train acc: 0.596000; val_acc: 0.572000
hidden=500 lr=0.1 reg=0.01 validation=0.583000
(Iteration 1 / 6125) loss: 2.492701
(Epoch 0 / 25) train acc: 0.105000; val_acc: 0.117000
(Epoch 1 / 25) train acc: 0.506000; val_acc: 0.513000
(Epoch 2 / 25) train acc: 0.534000; val_acc: 0.533000
(Epoch 3 / 25) train acc: 0.557000; val_acc: 0.546000
(Epoch 4 / 25) train acc: 0.525000; val_acc: 0.557000
(Iteration 1001 / 6125) loss: 1.291992
(Epoch 5 / 25) train acc: 0.574000; val_acc: 0.558000
(Epoch 6 / 25) train acc: 0.591000; val_acc: 0.557000
(Epoch 7 / 25) train acc: 0.567000; val_acc: 0.566000
(Epoch 8 / 25) train acc: 0.625000; val_acc: 0.571000
(Iteration 2001 / 6125) loss: 1.200354
(Epoch 9 / 25) train acc: 0.596000; val_acc: 0.566000
(Epoch 10 / 25) train acc: 0.632000; val_acc: 0.577000
(Epoch 11 / 25) train acc: 0.609000; val_acc: 0.570000

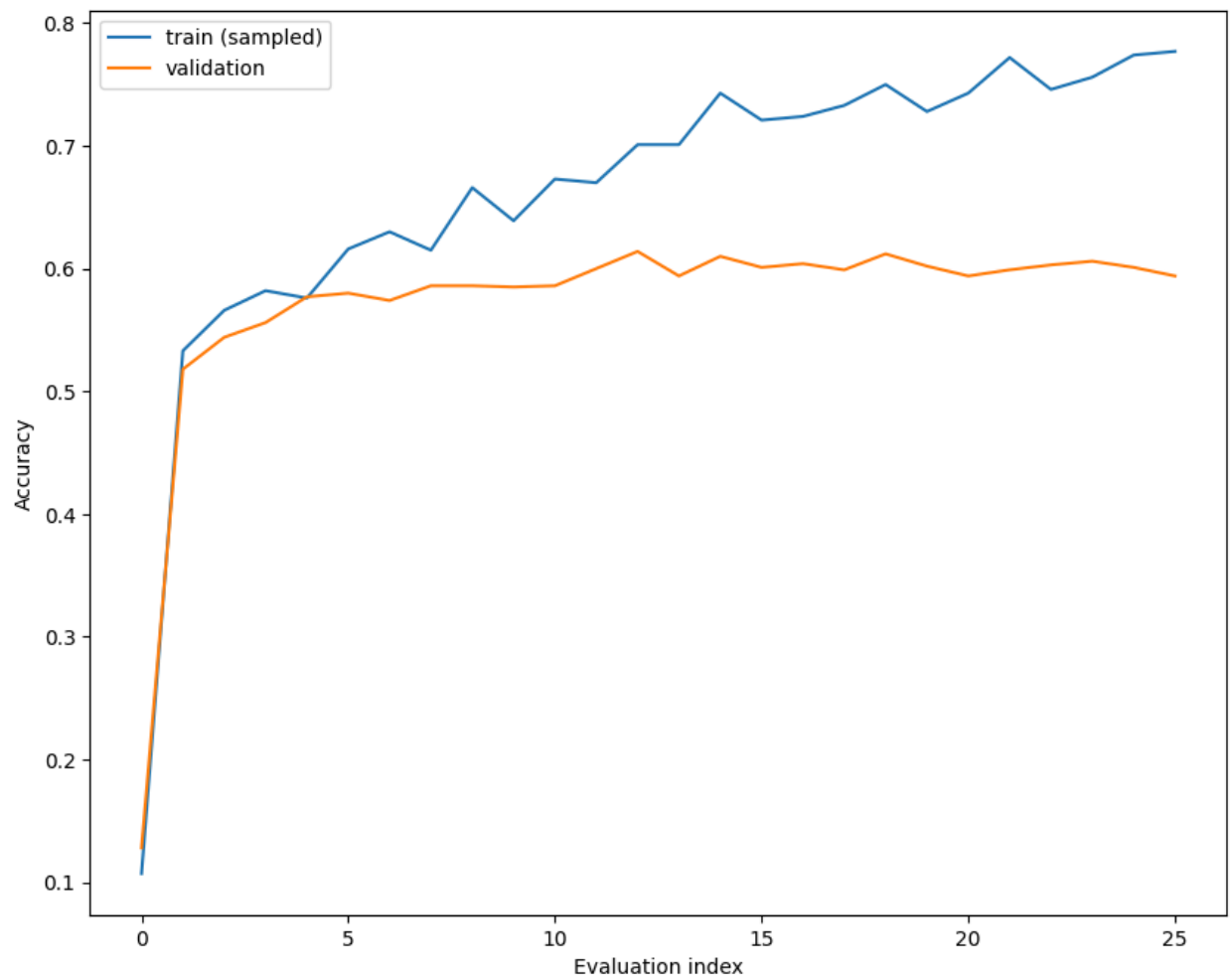
```

```

(Epoch 12 / 25) train acc: 0.636000; val_acc: 0.585000
(Iteration 3001 / 6125) loss: 1.290178
(Epoch 13 / 25) train acc: 0.639000; val_acc: 0.580000
(Epoch 14 / 25) train acc: 0.666000; val_acc: 0.582000
(Epoch 15 / 25) train acc: 0.645000; val_acc: 0.582000
(Epoch 16 / 25) train acc: 0.637000; val_acc: 0.580000
(Iteration 4001 / 6125) loss: 1.008847
(Epoch 17 / 25) train acc: 0.662000; val_acc: 0.584000
(Epoch 18 / 25) train acc: 0.684000; val_acc: 0.580000
(Epoch 19 / 25) train acc: 0.655000; val_acc: 0.584000
(Epoch 20 / 25) train acc: 0.664000; val_acc: 0.577000
(Iteration 5001 / 6125) loss: 1.141522
(Epoch 21 / 25) train acc: 0.693000; val_acc: 0.591000
(Epoch 22 / 25) train acc: 0.661000; val_acc: 0.596000
(Epoch 23 / 25) train acc: 0.678000; val_acc: 0.590000
(Epoch 24 / 25) train acc: 0.693000; val_acc: 0.586000
(Iteration 6001 / 6125) loss: 1.057233
(Epoch 25 / 25) train acc: 0.679000; val_acc: 0.592000
hidden=500 lr=0.05 reg=0.001 validation=0.596000
Best neural network validation accuracy: 0.614

```





```
In [11]: # Run your best neural net classifier on the test set. You should be able
# to get more than 58% accuracy. It is also possible to get >60% accuracy
# with careful tuning.
```

```
y_test_pred = np.argmax(best_net.loss(data['X_test']), axis=1)
test_acc = (y_test_pred == data['y_test']).mean()
print(test_acc)
```

0.571

```
In [12]: # Save best model
best_net.save("best_two_layer_net_features.npy")
```

best_two_layer_net_features.npy
saved.

```
In [ ]:
```

```
In [1]: # Local PyCharm / Jupyter setup
from local_setup import prepare
prepare()
```

Project: E:\Deeplearning\111\assignment1_colab\assignment1
Python: D:\Wei\python_3.12\python.exe
CIFAR-10 ready.

Multi-Layer Fully Connected Network

In this exercise, you will implement a fully connected network with an arbitrary number of hidden layers.

```
In [2]: # from google.colab import drive
# drive.mount('/content/drive')
```

Read through the `FullyConnectedNet` class in the file `cs231n/classifiers/fc_net.py`.

Implement the network initialization, forward pass, and backward pass. Throughout this assignment, you will be implementing layers in `cs231n/layers.py`. You can re-use your implementations for `affine_forward`, `affine_backward`, `relu_forward`, `relu_backward`, and `softmax_loss` from before. For right now, don't worry about implementing dropout or batch/layer normalization yet, as you will add those features later.

```
In [3]: # Setup cell.
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient, eval_numerical_gradient_array
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0) # Set default size of plots.
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

```
In [4]: # Load the (preprocessed) CIFAR-10 data.
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print(f"{k}: {v.shape}")
```

```
E:\Deeplearning\111\assignment1_colab\assignment1\cs231n\data_utils.py:16: VisibleDeprecationWarning: dtype
pe(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple t
o create a subarray type? (Deprecated NumPy 2.4)
    return pickle.load(f, encoding="latin1")
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

Initial Loss and Gradient Check

As a sanity check, run the following to check the initial loss and to gradient check the network both with and without regularization. This is a good way to see if the initial losses seem reasonable.

For gradient checking, you should expect to see errors around $1e-7$ or less.

```
In [5]: np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for reg in [0, 3.14]:
    print("Running check with reg = ", reg)
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
        weight_scale=5e-2,
        dtype=np.float64
    )

    loss, grads = model.loss(X, y)
    print("Initial loss: ", loss)

    # Most of the errors should be on the order of e-7 or smaller.
    # NOTE: It is fine however to see an error for W2 on the order of e-5
    # for the check when reg = 0.0
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name], verbose=False, h=1e-5)
        print(f"{name} relative error: {rel_error(grad_num, grads[name])}")
```

```
Running check with reg = 0
Initial loss: 2.3004790897684924
W1 relative error: 1.4839894098713283e-07
W2 relative error: 2.2120479295080622e-05
W3 relative error: 3.527252852151791e-07
b1 relative error: 5.376386228531692e-09
b2 relative error: 2.085654276112763e-09
b3 relative error: 5.7957243458479405e-11
Running check with reg = 3.14
Initial loss: 7.052114776533016
W1 relative error: 7.355058816898759e-09
W2 relative error: 6.86942277940646e-08
W3 relative error: 3.483989247437803e-08
b1 relative error: 1.4752427965311745e-08
b2 relative error: 1.7223751746766738e-09
b3 relative error: 1.801765144951982e-10
```

As another sanity check, make sure your network can overfit on a small dataset of 50 images. First, we will try a three-layer network with 100 units in each hidden layer. In the following cell, tweak the **learning rate** and **weight initialization scale** to overfit and achieve 100% training accuracy within 20 epochs.

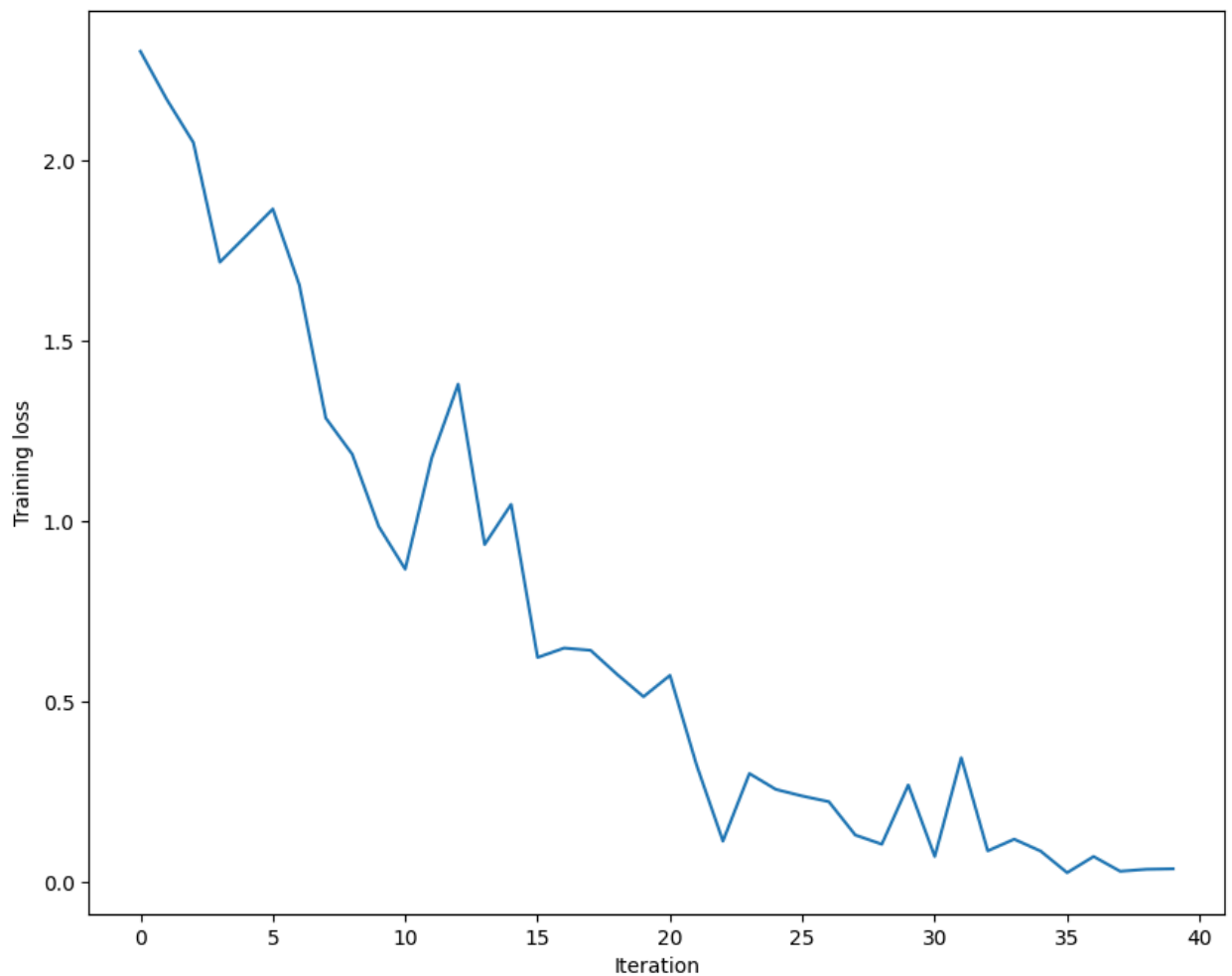
```
In [6]: # Adjust only initialization scale and learning rate; keep 50 samples and 20 epochs.
small_data = {'X_train': data['X_train'][:50], 'y_train': data['y_train'][:50],
              'X_val': data['X_val'], 'y_val': data['y_val']}
overfit_results = []
for weight_scale, learning_rate in [(0.01, 0.01), (0.01, 0.005), (0.02, 0.001), (0.05, 0.001)]:
    np.random.seed(231)
    model = FullyConnectedNet([100, 100], weight_scale=weight_scale, dtype=np.float64)
    solver = Solver(model, small_data, num_epochs=20, batch_size=25,
                    update_rule='sgd', optim_config={'learning_rate': learning_rate}, verbose=False)
```

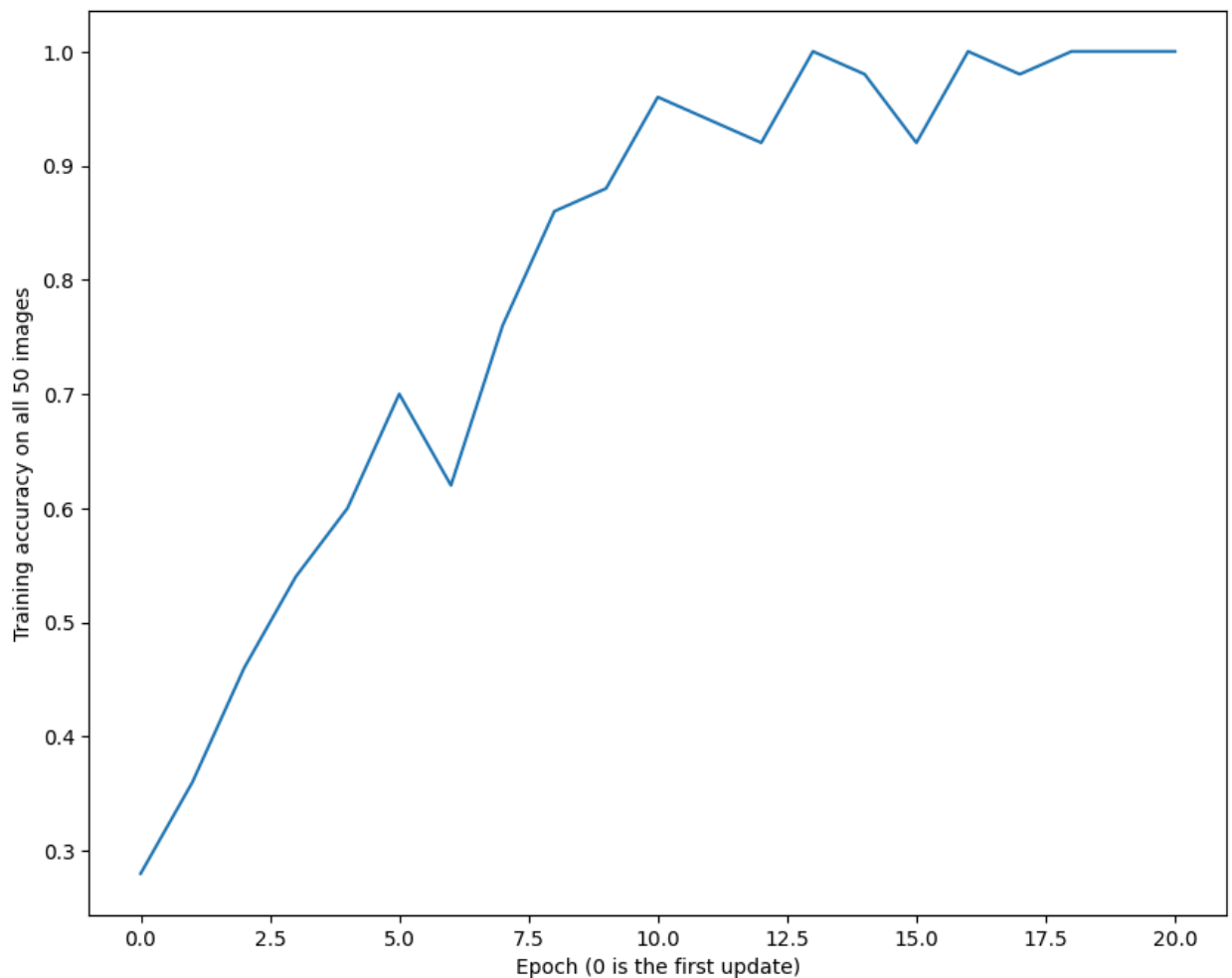
```

solver.train()
peak_train = max(solver.train_acc_history)
overfit_results.append((weight_scale, learning_rate, peak_train))
print('scale=%g lr=%g peak training accuracy=%f' % (weight_scale, learning_rate, peak_train))
if peak_train == 1.0:
    break
assert peak_train == 1.0, 'Initialization and learning rate need further tuning.'
print('First epoch reaching 100%:', solver.train_acc_history.index(1.0))
plt.plot(solver.loss_history)
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()
plt.plot(solver.train_acc_history)
plt.xlabel('Epoch (0 is the first update)')
plt.ylabel('Training accuracy on all 50 images')
plt.show()

```

scale=0.01 lr=0.01 peak training accuracy=1.000000
First epoch reaching 100%: 13

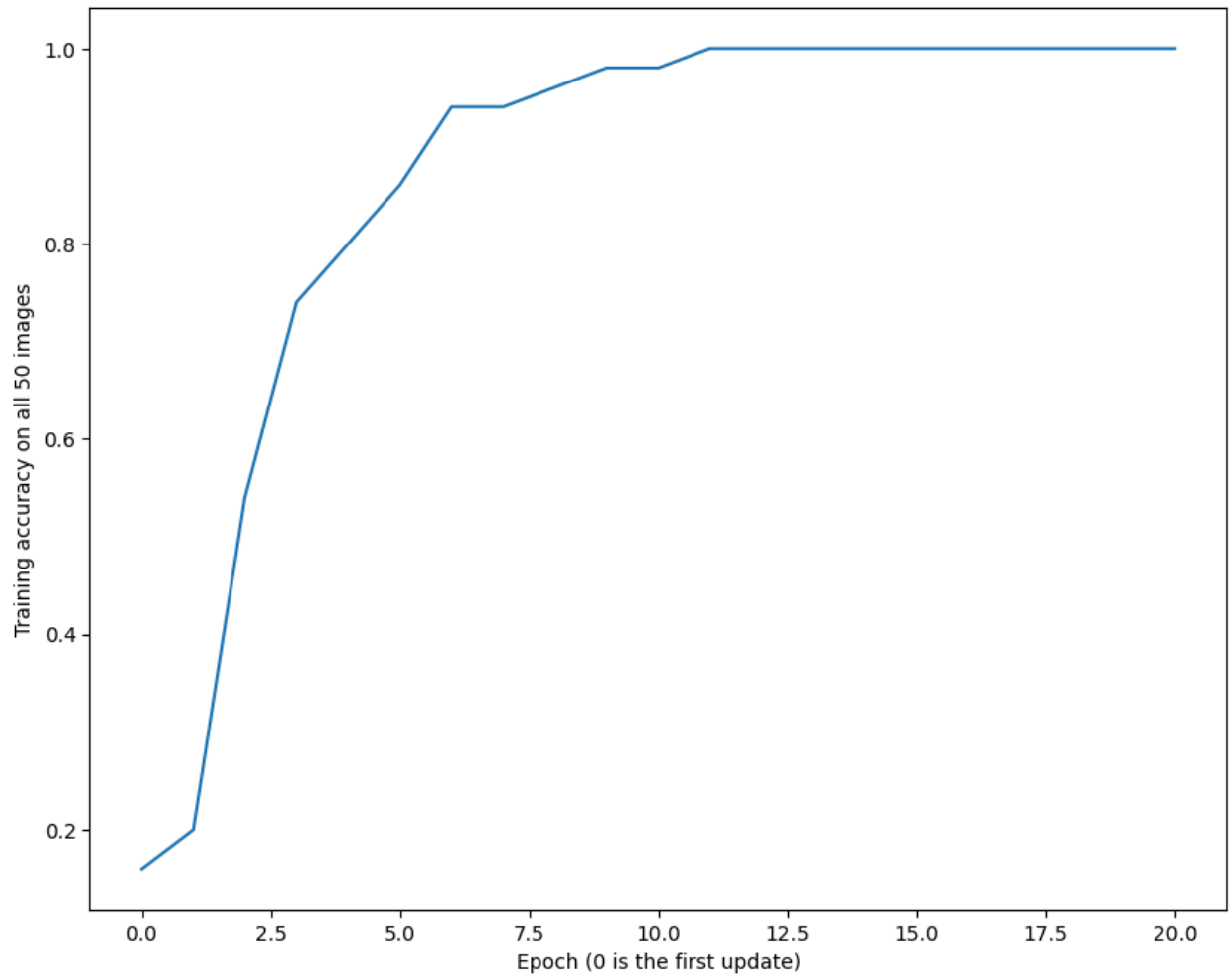
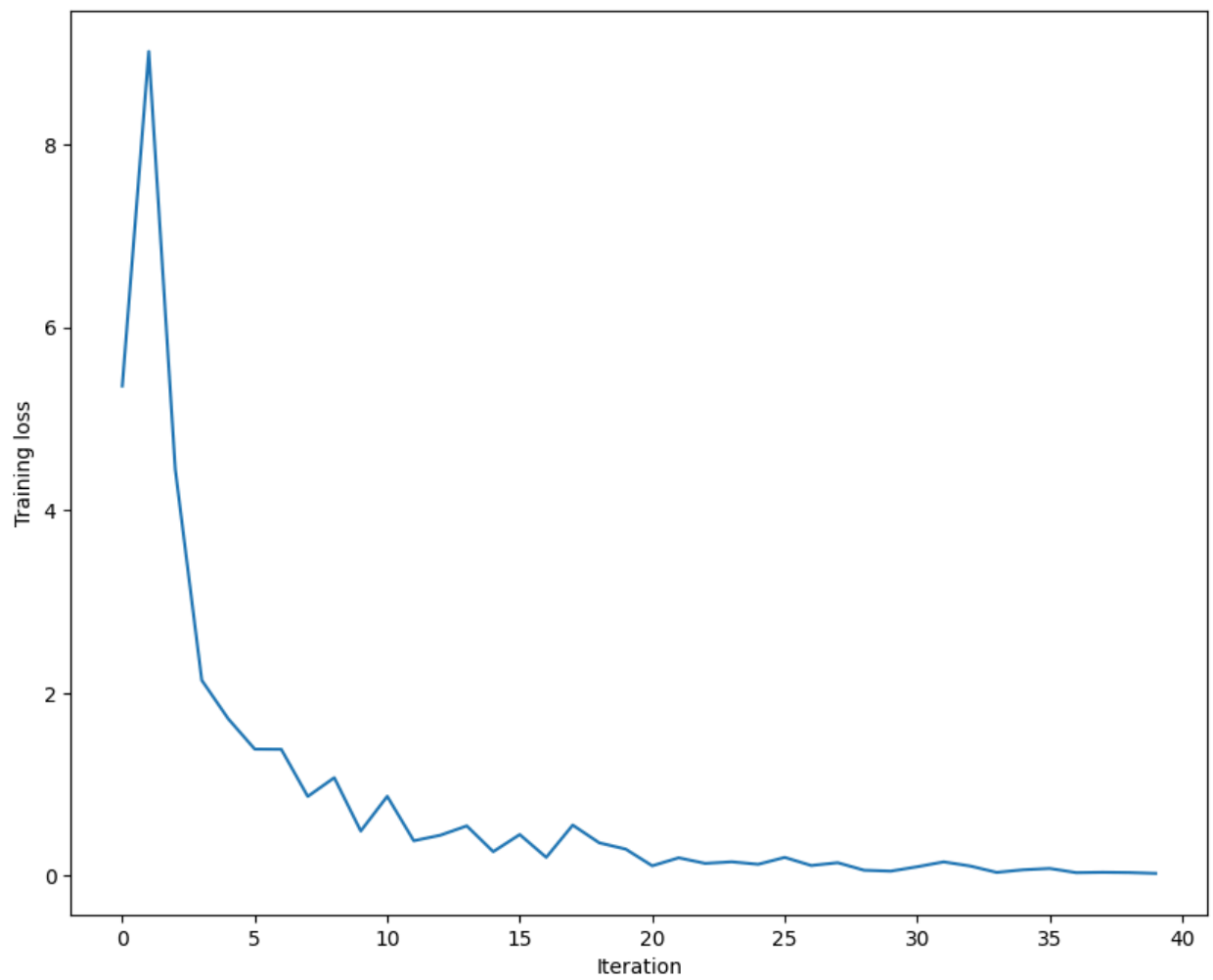




Now, try to use a five-layer network with 100 units on each layer to overfit on 50 training examples. Again, you will have to adjust the learning rate and weight initialization scale, but you should be able to achieve 100% training accuracy within 20 epochs.

```
In [7]: # Adjust only initialization scale and learning rate; keep 50 samples and 20 epochs.
small_data = {'X_train': data['X_train'][:50], 'y_train': data['y_train'][:50],
              'X_val': data['X_val'], 'y_val': data['y_val']}
overfit_results = []
for weight_scale, learning_rate in [(0.05, 0.01), (0.05, 0.005), (0.1, 0.001), (0.05, 0.001), (0.1,
0.005), (0.03, 0.01)]:
    np.random.seed(231)
    model = FullyConnectedNet([100, 100, 100, 100], weight_scale=weight_scale, dtype=np.float64)
    solver = Solver(model, small_data, num_epochs=20, batch_size=25,
                    update_rule='sgd', optim_config={'learning_rate': learning_rate}, verbose=False)
    solver.train()
    peak_train = max(solver.train_acc_history)
    overfit_results.append((weight_scale, learning_rate, peak_train))
    print('scale=%g lr=%g peak training accuracy=%f' % (weight_scale, learning_rate, peak_train))
    if peak_train == 1.0:
        break
assert peak_train == 1.0, 'Initialization and learning rate need further tuning.'
print('First epoch reaching 100%:', solver.train_acc_history.index(1.0))
plt.plot(solver.loss_history)
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()
plt.plot(solver.train_acc_history)
plt.xlabel('Epoch (0 is the first update)')
plt.ylabel('Training accuracy on all 50 images')
plt.show()
```

```
scale=0.05 lr=0.01 peak training accuracy=1.000000
First epoch reaching 100%: 11
```



Inline Question 1:

Did you notice anything about the comparative difficulty of training the three-layer network vs. training the five-layer network? In particular, based on your experience, which network seemed more sensitive to the initialization scale? Why do you think that is the case?

Answer:

With the selected settings, the three-layer network used initialization scale 0.01 and learning rate 0.01 and first reached 100% training accuracy at epoch 13. The five-layer network used scale 0.05 and the same learning rate and reached 100% at epoch 11. Thus, after tuning, the deeper network did not take more epochs in this run. These two successful runs alone do not measure sensitivity systematically. However, the deeper network is expected to be more sensitive to initialization because activations and backpropagated gradients pass through more weight matrices and ReLU gates: repeated small scales can attenuate signals, while excessive scales can amplify them. This motivates tuning initialization jointly with the learning rate; a larger suitable scale here should not be interpreted as a universal rule.

Update rules

So far we have used vanilla stochastic gradient descent (SGD) as our update rule. More sophisticated update rules can make it easier to train deep networks. We will implement a few of the most commonly used update rules and compare them to vanilla SGD.

SGD+Momentum

Stochastic gradient descent with momentum is a widely used update rule that tends to make deep networks converge faster than vanilla stochastic gradient descent. See the Momentum Update section at <http://cs231n.github.io/neural-networks-3/#sgd> for more information.

Open the file `cs231n/optim.py` and read the documentation at the top of the file to make sure you understand the API. Implement the SGD+momentum update rule in the function `sgd_momentum` and run the following to check your implementation. You should see errors less than $e-8$.

```
In [8]: from cs231n.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {"learning_rate": 1e-3, "velocity": v}
```

```

next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
    [ 0.1406,    0.20738947,  0.27417895,  0.34096842,  0.40775789],
    [ 0.47454737, 0.54133684,  0.60812632,  0.67491579,  0.74170526],
    [ 0.80849474, 0.87528421,  0.94207368,  1.00886316,  1.07565263],
    [ 1.14244211, 1.20923158,  1.27602105,  1.34281053,  1.4096    ]])
expected_velocity = np.asarray([
    [ 0.5406,    0.55475789,  0.56891579,  0.58307368,  0.59723158],
    [ 0.61138947, 0.62554737,  0.63970526,  0.65386316,  0.66802105],
    [ 0.68217895, 0.69633684,  0.71049474,  0.72465263,  0.73881053],
    [ 0.75296842, 0.76712632,  0.78128421,  0.79544211,  0.8096    ]])

# Should see relative errors around e-8 or less
print("next_w error: ", rel_error(next_w, expected_next_w))
print("velocity error: ", rel_error(expected_velocity, config["velocity"]))

```

```

next_w error:  8.882347033505819e-09
velocity error: 4.269287743278663e-09

```

Once you have done so, run the following to train a six-layer network with both SGD and SGD+momentum. You should see the SGD+momentum update rule converge faster.

```

In [9]: num_train = 4000
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    np.random.seed(231)
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"loss_{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"train_acc_{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"val_acc_{update_rule}")

for ax in axes:
    ax.legend(loc="best", ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

```

```
plt.show()
```

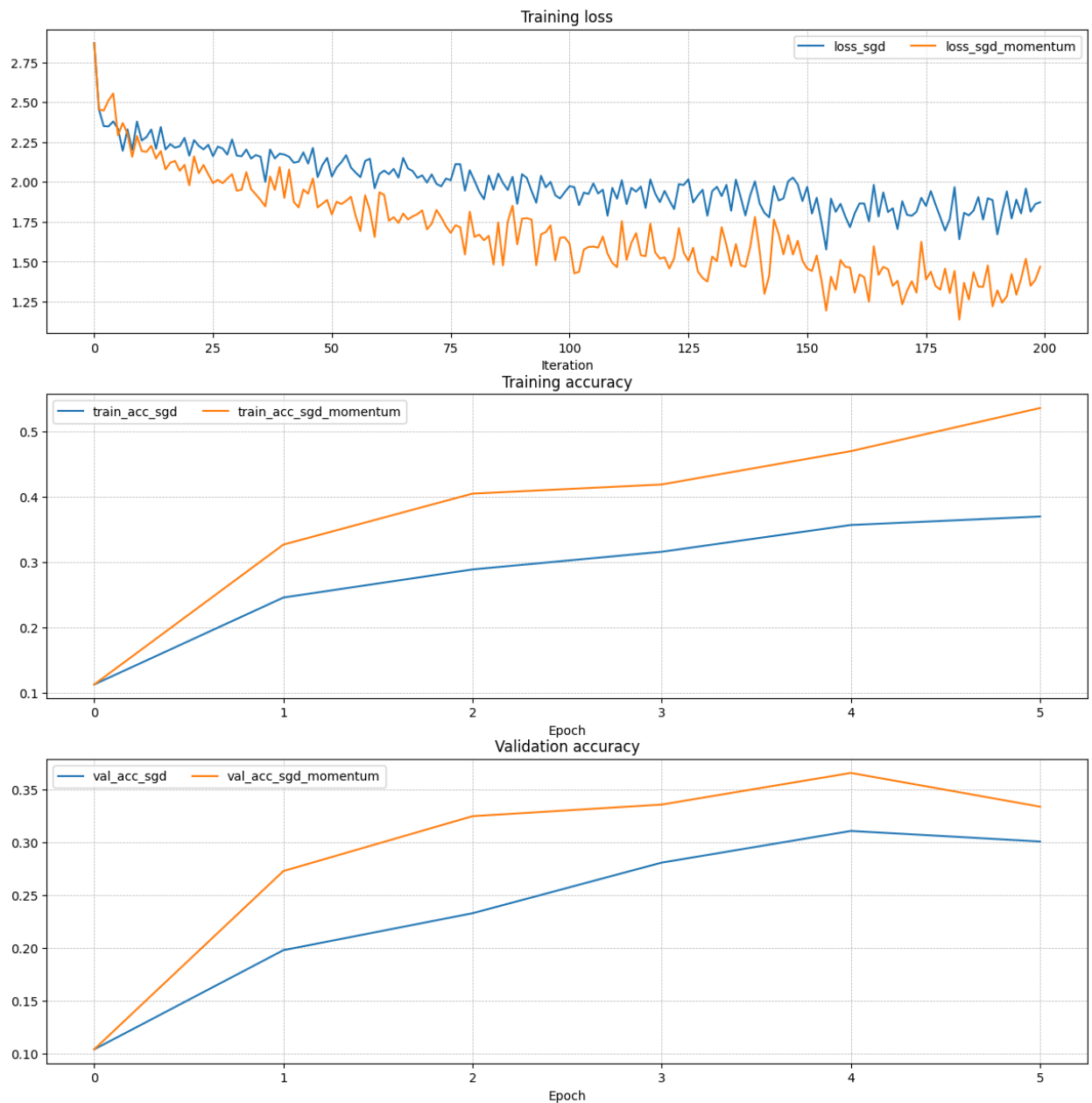
Running with

sgd

```
(Iteration 1 / 200) loss: 2.870056
(Epoch 0 / 5) train acc: 0.113000; val_acc: 0.104000
(Iteration 11 / 200) loss: 2.260219
(Iteration 21 / 200) loss: 2.164584
(Iteration 31 / 200) loss: 2.164986
(Epoch 1 / 5) train acc: 0.246000; val_acc: 0.198000
(Iteration 41 / 200) loss: 2.171974
(Iteration 51 / 200) loss: 2.034330
(Iteration 61 / 200) loss: 2.049563
(Iteration 71 / 200) loss: 1.996434
(Epoch 2 / 5) train acc: 0.289000; val_acc: 0.233000
(Iteration 81 / 200) loss: 2.008327
(Iteration 91 / 200) loss: 2.048697
(Iteration 101 / 200) loss: 1.973643
(Iteration 111 / 200) loss: 1.894510
(Epoch 3 / 5) train acc: 0.316000; val_acc: 0.281000
(Iteration 121 / 200) loss: 1.944060
(Iteration 131 / 200) loss: 1.942249
(Iteration 141 / 200) loss: 1.864212
(Iteration 151 / 200) loss: 1.969626
(Epoch 4 / 5) train acc: 0.357000; val_acc: 0.311000
(Iteration 161 / 200) loss: 1.804148
(Iteration 171 / 200) loss: 1.879613
(Iteration 181 / 200) loss: 1.767780
(Iteration 191 / 200) loss: 1.671593
(Epoch 5 / 5) train acc: 0.370000; val_acc: 0.301000
```

Running with sgd_momentum

```
(Iteration 1 / 200) loss: 2.870056
(Epoch 0 / 5) train acc: 0.113000; val_acc: 0.104000
(Iteration 11 / 200) loss: 2.193800
(Iteration 21 / 200) loss: 1.979138
(Iteration 31 / 200) loss: 1.944965
(Epoch 1 / 5) train acc: 0.327000; val_acc: 0.273000
(Iteration 41 / 200) loss: 1.900573
(Iteration 51 / 200) loss: 1.795972
(Iteration 61 / 200) loss: 1.934806
(Iteration 71 / 200) loss: 1.703238
(Epoch 2 / 5) train acc: 0.405000; val_acc: 0.325000
(Iteration 81 / 200) loss: 1.655955
(Iteration 91 / 200) loss: 1.769628
(Iteration 101 / 200) loss: 1.613561
(Iteration 111 / 200) loss: 1.466805
(Epoch 3 / 5) train acc: 0.419000; val_acc: 0.336000
(Iteration 121 / 200) loss: 1.527865
(Iteration 131 / 200) loss: 1.532239
(Iteration 141 / 200) loss: 1.564208
(Iteration 151 / 200) loss: 1.457400
(Epoch 4 / 5) train acc: 0.470000; val_acc: 0.366000
(Iteration 161 / 200) loss: 1.306821
(Iteration 171 / 200) loss: 1.233548
(Iteration 181 / 200) loss: 1.304058
(Iteration 191 / 200) loss: 1.321157
(Epoch 5 / 5) train acc: 0.536000; val_acc: 0.334000
```



RMSProp and Adam

RMSProp [1] and Adam [2] are update rules that set per-parameter learning rates by using a running average of the second moments of gradients.

In the file `cs231n/optim.py`, implement the RMSProp update rule in the `rmsprop` function and implement the Adam update rule in the `adam` function, and check your implementations using the tests below.

NOTE: Please implement the *complete* Adam update rule (with the bias correction mechanism), not the first simplified version mentioned in the course notes.

[1] Tijmen Tieleman and Geoffrey Hinton. "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude." COURSE: Neural Networks for Machine Learning 4 (2012).

[2] Diederik Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015.

```
In [10]: # Test RMSProp implementation
from cs231n.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
    [-0.132737, -0.08078555, -0.02881884, 0.02316247, 0.07515774],
    [0.12716641, 0.17918792, 0.23122175, 0.28326742, 0.33532447],
    [0.38739248, 0.43947102, 0.49155973, 0.54365823, 0.59576619]])
expected_cache = np.asarray([
    [0.5976, 0.6126277, 0.6277108, 0.64284931, 0.65804321],
    [0.67329252, 0.68859723, 0.70395734, 0.71937285, 0.73484377],
    [0.75037008, 0.7659518, 0.78158892, 0.79728144, 0.81302936],
    [0.82883269, 0.84469141, 0.86060554, 0.87657507, 0.8926 ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

next_w error: 9.524687511038133e-08
cache error: 2.6477955807156126e-09

```
In [11]: # Test Adam implementation
from cs231n.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
    [-0.1380274, -0.08544591, -0.03286534, 0.01971428, 0.0722929],
    [0.1248705, 0.17744702, 0.23002243, 0.28259667, 0.33516969],
    [0.38774145, 0.44031188, 0.49288093, 0.54544852, 0.59801459]])
expected_v = np.asarray([
    [0.69966, 0.68908382, 0.67851319, 0.66794809, 0.65738853],
    [0.64683452, 0.63628604, 0.6257431, 0.61520571, 0.60467385],
    [0.59414753, 0.58362676, 0.57311152, 0.56260183, 0.55209767],
    [0.54159906, 0.53110598, 0.52061845, 0.51013645, 0.49966 ]])
expected_m = np.asarray([
    [0.48, 0.49947368, 0.51894737, 0.53842105, 0.55789474],
    [0.57736842, 0.59684211, 0.61631579, 0.63578947, 0.65526316],
    [0.67473684, 0.69421053, 0.71368421, 0.73315789, 0.75263158],
    [0.77210526, 0.79157895, 0.81105263, 0.83052632, 0.85 ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))
```

next_w error: 1.1395691798535431e-07
v error: 4.208314038113071e-09
m error: 4.214963193114416e-09

Once you have debugged your RMSProp and Adam implementations, run the following to train a pair of deep networks using these new update rules:

```
In [12]: learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    np.random.seed(231)
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True
    )
    solvers[update_rule] = solver
    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"{update_rule}")

for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

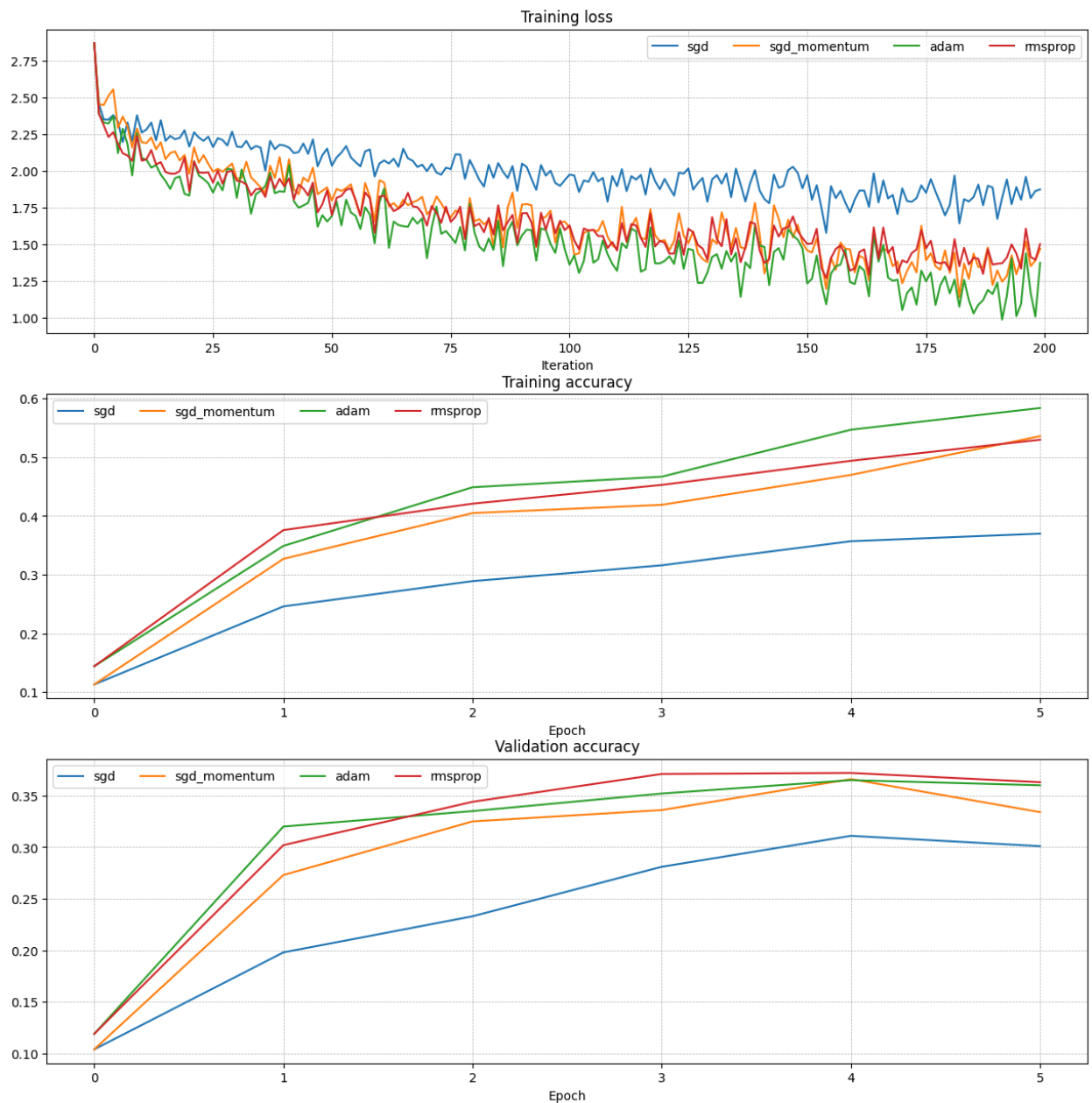
plt.show()
```

```
Running with adam
(Iteration 1 / 200) loss: 2.870056
(Epoch 0 / 5) train acc: 0.144000; val_acc: 0.119000
(Iteration 11 / 200) loss: 2.087330
(Iteration 21 / 200) loss: 1.830715
(Iteration 31 / 200) loss: 1.815505
(Epoch 1 / 5) train acc: 0.349000; val_acc: 0.320000
(Iteration 41 / 200) loss: 1.856429
(Iteration 51 / 200) loss: 1.690585
(Iteration 61 / 200) loss: 1.783514
(Iteration 71 / 200) loss: 1.403502
(Epoch 2 / 5) train acc: 0.449000; val_acc: 0.335000
(Iteration 81 / 200) loss: 1.580006
(Iteration 91 / 200) loss: 1.560271
(Iteration 101 / 200) loss: 1.360471
(Iteration 111 / 200) loss: 1.318104
(Epoch 3 / 5) train acc: 0.467000; val_acc: 0.352000
(Iteration 121 / 200) loss: 1.385121
(Iteration 131 / 200) loss: 1.412997
(Iteration 141 / 200) loss: 1.490376
(Iteration 151 / 200) loss: 1.231833
(Epoch 4 / 5) train acc: 0.547000; val_acc: 0.365000
(Iteration 161 / 200) loss: 1.227751
(Iteration 171 / 200) loss: 1.050260
(Iteration 181 / 200) loss: 1.164228
(Iteration 191 / 200) loss: 1.237766
```

(Epoch 5 / 5) train acc: 0.584000; val_acc: 0.360000

Running with rmsprop

(Iteration 1 / 200) loss: 2.870056
(Epoch 0 / 5) train acc: 0.144000; val_acc: 0.119000
(Iteration 11 / 200) loss: 2.068296
(Iteration 21 / 200) loss: 1.861133
(Iteration 31 / 200) loss: 1.931985
(Epoch 1 / 5) train acc: 0.376000; val_acc: 0.302000
(Iteration 41 / 200) loss: 1.893628
(Iteration 51 / 200) loss: 1.699124
(Iteration 61 / 200) loss: 1.823108
(Iteration 71 / 200) loss: 1.625993
(Epoch 2 / 5) train acc: 0.421000; val_acc: 0.344000
(Iteration 81 / 200) loss: 1.622576
(Iteration 91 / 200) loss: 1.709796
(Iteration 101 / 200) loss: 1.626642
(Iteration 111 / 200) loss: 1.455891
(Epoch 3 / 5) train acc: 0.453000; val_acc: 0.371000
(Iteration 121 / 200) loss: 1.509939
(Iteration 131 / 200) loss: 1.683399
(Iteration 141 / 200) loss: 1.456053
(Iteration 151 / 200) loss: 1.501451
(Epoch 4 / 5) train acc: 0.494000; val_acc: 0.372000
(Iteration 161 / 200) loss: 1.333687
(Iteration 171 / 200) loss: 1.389087
(Iteration 181 / 200) loss: 1.323181
(Iteration 191 / 200) loss: 1.366614
(Epoch 5 / 5) train acc: 0.530000; val_acc: 0.363000



Inline Question 2:

AdaGrad, like Adam, is a per-parameter optimization method that uses the following update rule:

```
cache += dw**2
w += - learning_rate * dw / (np.sqrt(cache) + eps)
```

John notices that when he was training a network with AdaGrad that the updates became very small, and that his network was learning slowly. Using your knowledge of the AdaGrad update rule, why do you think the updates would become very small? Would Adam have the same issue?

Answer:

AdaGrad accumulates squared gradients without forgetting, so its denominator is nondecreasing and its effective per-parameter learning rates can become very small. Adam instead uses exponential moving averages of gradients and squared gradients, together with bias correction, so old gradients decay in influence. It does not have the same unavoidable cumulative-denominator growth, although small gradients or other optimization problems can still make its updates small.

Train a Good Model!

Train the best fully connected model that you can on CIFAR-10, storing your best model in the `best_model` variable. We require you to get at least 50% accuracy on the validation set using a fully connected network.

If you are careful it should be possible to get accuracies above 55%, but we don't require it for this part and won't assign extra credit for doing so. Later in the next assignment, we will ask you to train the best convolutional network that you can on CIFAR-10, and we would prefer that you spend your effort working on convolutional networks rather than fully connected networks.

Note: In the next assignment, you will learn techniques like BatchNormalization and Dropout which can help you train powerful models.

```
In [13]: best_model = None
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
configs = [(256, 128, 5e-4, 0.001), (256, 128, 3e-4, 0.01)]
best_val = -1
results = {}
for hidden, lr, reg in configs:
    np.random.seed(42)
    candidate = FullyConnectedNet(hidden, weight_scale=0.02, reg=reg, dtype=np.float32)
    candidate_solver = Solver(candidate, data, update_rule='adam',
                              optim_config={'learning_rate': lr}, lr_decay=0.95,
                              num_epochs=20, batch_size=200, print_every=1000)

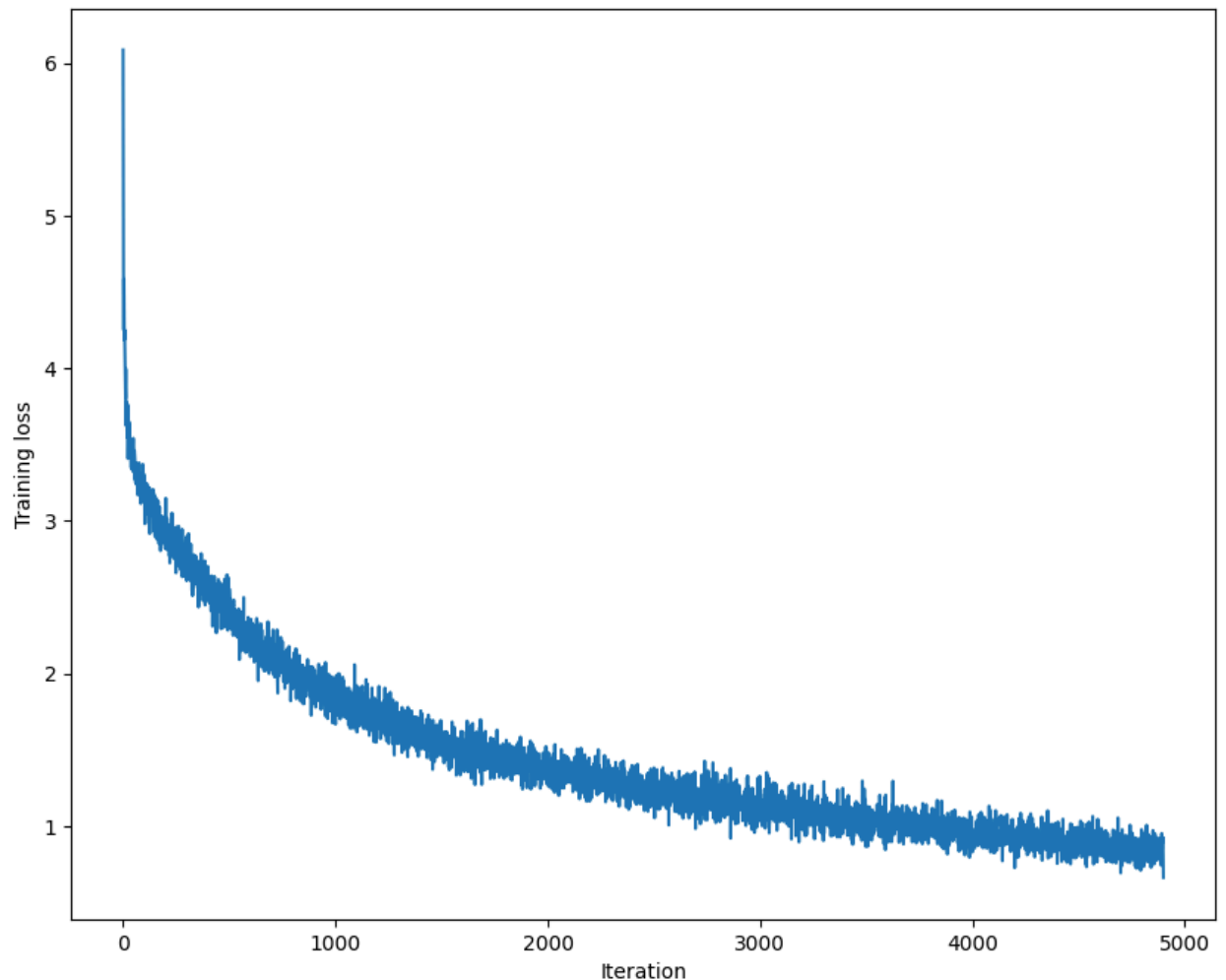
    candidate_solver.train()
    val_acc = candidate_solver.check_accuracy(data['X_val'], data['y_val'])
    results[(tuple(hidden), lr, reg)] = val_acc
    print('hidden=%s lr=%g reg=%g validation=%f' % (hidden, lr, reg, val_acc))
    if val_acc > best_val:
        best_val, best_model, best_solver = val_acc, candidate, candidate_solver
print('Best validation accuracy:', best_val)
assert best_val >= 0.50
best_model.save('best_fully_connected_net.npy')
plt.plot(best_solver.loss_history)
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()
# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
```

```
(Iteration 1 / 4900) loss: 4.607330
(Epoch 0 / 20) train acc: 0.169000; val_acc: 0.159000
(Epoch 1 / 20) train acc: 0.457000; val_acc: 0.420000
(Epoch 2 / 20) train acc: 0.517000; val_acc: 0.449000
(Epoch 3 / 20) train acc: 0.573000; val_acc: 0.482000
(Epoch 4 / 20) train acc: 0.544000; val_acc: 0.526000
(Iteration 1001 / 4900) loss: 1.269001
(Epoch 5 / 20) train acc: 0.578000; val_acc: 0.521000
(Epoch 6 / 20) train acc: 0.601000; val_acc: 0.523000
```

```

(Epoch 7 / 20) train acc: 0.630000; val_acc: 0.520000
(Epoch 8 / 20) train acc: 0.628000; val_acc: 0.512000
(Iteration 2001 / 4900) loss: 1.131947
(Epoch 9 / 20) train acc: 0.682000; val_acc: 0.515000
(Epoch 10 / 20) train acc: 0.682000; val_acc: 0.513000
(Epoch 11 / 20) train acc: 0.706000; val_acc: 0.544000
(Epoch 12 / 20) train acc: 0.728000; val_acc: 0.511000
(Iteration 3001 / 4900) loss: 0.988072
(Epoch 13 / 20) train acc: 0.753000; val_acc: 0.528000
(Epoch 14 / 20) train acc: 0.754000; val_acc: 0.531000
(Epoch 15 / 20) train acc: 0.747000; val_acc: 0.518000
(Epoch 16 / 20) train acc: 0.805000; val_acc: 0.510000
(Iteration 4001 / 4900) loss: 0.761345
(Epoch 17 / 20) train acc: 0.808000; val_acc: 0.520000
(Epoch 18 / 20) train acc: 0.804000; val_acc: 0.534000
(Epoch 19 / 20) train acc: 0.822000; val_acc: 0.518000
(Epoch 20 / 20) train acc: 0.857000; val_acc: 0.530000
hidden=[256, 128] lr=0.0005 reg=0.001 validation=0.544000
(Iteration 1 / 4900) loss: 6.085105
(Epoch 0 / 20) train acc: 0.141000; val_acc: 0.141000
(Epoch 1 / 20) train acc: 0.483000; val_acc: 0.432000
(Epoch 2 / 20) train acc: 0.506000; val_acc: 0.455000
(Epoch 3 / 20) train acc: 0.579000; val_acc: 0.502000
(Epoch 4 / 20) train acc: 0.561000; val_acc: 0.499000
(Iteration 1001 / 4900) loss: 1.667156
(Epoch 5 / 20) train acc: 0.571000; val_acc: 0.491000
(Epoch 6 / 20) train acc: 0.605000; val_acc: 0.505000
(Epoch 7 / 20) train acc: 0.669000; val_acc: 0.516000
(Epoch 8 / 20) train acc: 0.620000; val_acc: 0.544000
(Iteration 2001 / 4900) loss: 1.344287
(Epoch 9 / 20) train acc: 0.655000; val_acc: 0.517000
(Epoch 10 / 20) train acc: 0.662000; val_acc: 0.547000
(Epoch 11 / 20) train acc: 0.712000; val_acc: 0.522000
(Epoch 12 / 20) train acc: 0.701000; val_acc: 0.538000
(Iteration 3001 / 4900) loss: 1.235566
(Epoch 13 / 20) train acc: 0.737000; val_acc: 0.528000
(Epoch 14 / 20) train acc: 0.725000; val_acc: 0.535000
(Epoch 15 / 20) train acc: 0.768000; val_acc: 0.532000
(Epoch 16 / 20) train acc: 0.777000; val_acc: 0.525000
(Iteration 4001 / 4900) loss: 0.897173
(Epoch 17 / 20) train acc: 0.786000; val_acc: 0.544000
(Epoch 18 / 20) train acc: 0.773000; val_acc: 0.526000
(Epoch 19 / 20) train acc: 0.802000; val_acc: 0.523000
(Epoch 20 / 20) train acc: 0.823000; val_acc: 0.535000
hidden=[256, 128] lr=0.0003 reg=0.01 validation=0.547000
Best validation accuracy: 0.547
best_fully_connected_net.npy saved.

```



Test Your Model!

Run your best model on the validation and test sets. You should achieve at least 50% accuracy on the validation set and the test set.

```
In [14]: y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Validation set accuracy: 0.547
Test set accuracy: 0.528
```